TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

Colivi: Una app de alojamientos para estudiantes.



Curso: 2026/2027

Alumno/a:
Víctor Vallejo Uroz

Director/es:
Luis Iribarne Martínez

Texto de la dedicatoria

ÍNDICE GENERAL

Página

Resumen y Abstract XIV

1. Introducción 1

1.1. Justificación . . . 1
1.2. Objetivos Y Estructura . . . 2
1.3. Planificacion . . . 3
1.4. Apps Similares . . . 5
1.5. Contenido de la memoria . . . 8

2. Herramientas y metodologías 11

2.1. Metodologías usadas . . . 12
2.1.1. Revisión de enfoques metodológicos . . . 12
2.1.2. Análisis comparativo y selección para el proyecto . . . 13
2.2. Herramientas usadas . . . 15
2.2.1. Entornos de desarrollo y asistencia técnica . . . 15
2.2.2. Lenguajes, librerías y servicios . . . 15
2.2.3. Almacenamiento y persistencia de datos . . . 16
2.2.4. Control de versiones, infraestructura y utilidades auxiliares . . . 17

3. Análisis funcional 19

3.1. Diagramas de casos de uso . . . 19
3.1.1. Actores del sistema . . . 19
3.1.2. Especificación detallada de casos de uso . . . 21
3.1.3. Paquete 1: Gestión de identidad, sesión y perfil . . . 21
3.1.4. Paquete 2: Gestión de inmuebles, publicaciones y confianza . . . 28
3.1.4.1. Exploración de alojamientos, recomendaciones y sistema de confianza 28
3.1.4.2. Gestión de inmuebles y ciclo de publicación del anfitrión . . . 31
3.1.5. Paquete 3: Ciclo de vida de reservas y depósitos . . . 35
3.1.6. Paquete 4: Comunidad de convivencia y espacio del hogar . . . 38
3.1.7. Paquete 5: Gestión financiera compartida . . . 43

3.1.8. Paquete 6: Motor de tareas domésticas y sistema de puntos 46
3.1.9. Paquete 7: Mensajería contextual 49
3.1.10. Paquete 8: Asistente inteligente y orquestación de herramientas 51
3.1.11. Paquete 9: Panel de administración, gobernanza y moderación global 55

4. Apartados del CUERPO DEL DOCUMENTO **61**

5. Apartado BIBLIOGRAFIA **65**
5.1. Como citar 65
5.2. Apartado referencias 65
5.3. Archivos .bib 66
BIBLIOGRAFÍA 66

A. Contenido de una entrada en .bib **69**

B. Tipos de referencias **71**

ÍNDICE DE FIGURAS

1.1. Estructura técnica global y arquitectura de la plataforma Colivi 4
1.2. Planificación Temporal General 4
1.3. Planificación Temporal de la implementación 5
1.4. Interfaces de las principales soluciones analizadas en el mercado 7

2.1. Diferencia de éxito entre metodologías ágiles y tradicionales en cascada 12
2.2. Tablero Kanban representativo del flujo de desarrollo ágil 13
2.3. Ciclo iterativo de la metodología TDD (*Test-Driven Development*) 14
2.4. Entornos de desarrollo utilizados en el proyecto. Elaboración propia 15
2.5. Sistema gestor de bases de datos PostgreSQL 16
2.6. Herramientas auxiliares empleadas para diseño de interfaz, modelado y verificación de API 17

3.1. Jerarquía y relaciones de herencia entre los actores del sistema. Elaboración propia 20
3.2. Diagrama de casos de uso del módulo de identidad, sesión y perfil 21
3.3. Diagrama de casos de uso de exploración, recomendaciones y reseñas 28
3.4. Diagrama de casos de uso de gestión de inmuebles y publicaciones del anfitrión 32
3.5. Diagrama de casos de uso del ciclo de vida de reservas 35
3.6. Diagrama de casos de uso de comunidad de convivencia y hogar 38
3.7. Diagrama de casos de uso de gestión financiera compartida 43
3.8. Diagrama de casos de uso de tareas domésticas y puntos 47
3.9. Diagrama de casos de uso de mensajería contextual 49
3.10. Diagrama de casos de uso de asistente inteligente y orquestación de herramientas 52
3.11. Diagrama de casos de uso de moderación y administración global 55

4.1. Estructura clásica de un articulo de investigación 62
4.2. Estructura de documentos de un proyecto técnico 62
4.3. Planificación del proyecto 63

ÍNDICE DE TABLAS

1.1. Comparativa de características entre plataformas analizadas y Colivi 6

3.1. Especificación del caso de uso CU01. 22
3.2. Especificación del caso de uso CU02. 22
3.3. Especificación del caso de uso CU03. 23
3.4. Especificación del caso de uso CU04. 23
3.5. Especificación del caso de uso CU05. 23
3.6. Especificación del caso de uso CU06. 24
3.7. Especificación del caso de uso CU07. 24
3.8. Especificación del caso de uso CU08. 25
3.9. Especificación del caso de uso CU09. 25
3.10. Especificación del caso de uso CU10. 25
3.11. Especificación del caso de uso CU11. 26
3.12. Especificación del caso de uso CU12. 26
3.13. Especificación del caso de uso CU13. 27
3.14. Especificación del caso de uso CU14. 27
3.15. Especificación del caso de uso CU15. 28
3.16. Especificación del caso de uso CU16. 29
3.17. Especificación del caso de uso CU17. 29
3.18. Especificación del caso de uso CU18. 29
3.19. Especificación del caso de uso CU19. 30
3.20. Especificación del caso de uso CU20. 30
3.21. Especificación del caso de uso CU21. 30
3.22. Especificación del caso de uso CU22. 31
3.23. Especificación del caso de uso CU23. 31
3.24. Especificación del caso de uso CU24. 32
3.25. Especificación del caso de uso CU25. 32
3.26. Especificación del caso de uso CU26. 33

3.27. Especificación del caso de uso CU27. 33
3.28. Especificación del caso de uso CU28. 34
3.29. Especificación del caso de uso CU29. 34
3.30. Especificación del caso de uso CU30. 34
3.31. Especificación del caso de uso CU31. 35
3.32. Especificación del caso de uso CU32. 35
3.33. Especificación del caso de uso CU33. 36
3.34. Especificación del caso de uso CU34. 36
3.35. Especificación del caso de uso CU35. 36
3.36. Especificación del caso de uso CU36. 37
3.37. Especificación del caso de uso CU37. 37
3.38. Especificación del caso de uso CU38. 37
3.39. Especificación del caso de uso CU39 39
3.40. Especificación del caso de uso CU40 39
3.41. Especificación del caso de uso CU41 39
3.42. Especificación del caso de uso CU42 40
3.43. Especificación del caso de uso CU43 40
3.44. Especificación del caso de uso CU44 40
3.45. Especificación del caso de uso CU45 41
3.46. Especificación del caso de uso CU46 41
3.47. Especificación del caso de uso CU47 42
3.48. Especificación del caso de uso CU48 42
3.49. Especificación del caso de uso CU49 42
3.50. Especificación del caso de uso CU50 43
3.51. Especificación del caso de uso CU51 44
3.52. Especificación del caso de uso CU52 44
3.53. Especificación del caso de uso CU53 45
3.54. Especificación del caso de uso CU54 45
3.55. Especificación del caso de uso CU55 46
3.56. Especificación del caso de uso CU56 46
3.57. Especificación del caso de uso CU57 47
3.58. Especificación del caso de uso CU58 47
3.59. Especificación del caso de uso CU59 48
3.60. Especificación del caso de uso CU60 48
3.61. Especificación del caso de uso CU61 48

3.62. Especificación del caso de uso CU62 49
3.63. Especificación del caso de uso CU63 49
3.64. Especificación del caso de uso CU64 50
3.65. Especificación del caso de uso CU65 50
3.66. Especificación del caso de uso CU66 50
3.67. Especificación del caso de uso CU67 51
3.68. Especificación del caso de uso CU68 51
3.69. Especificación del caso de uso CU69 52
3.70. Especificación del caso de uso CU70 52
3.71. Especificación del caso de uso CU71 53
3.72. Especificación del caso de uso CU72 53
3.73. Especificación del caso de uso CU73 54
3.74. Especificación del caso de uso CU74 54
3.75. Especificación del caso de uso CU75 55
3.76. Especificación del caso de uso CU76 56
3.77. Especificación del caso de uso CU77 56
3.78. Especificación del caso de uso CU78 56
3.79. Especificación del caso de uso CU79 57
3.80. Especificación del caso de uso CU80 57
3.81. Especificación del caso de uso CU81 57
3.82. Especificación del caso de uso CU82 58
3.83. Especificación del caso de uso CU83 58
3.84. Especificación del caso de uso CU84 58
3.85. Especificación del caso de uso CU85 59
3.86. Especificación del caso de uso CU86 59

A.1. Campos de una entrada en archivo de bibliografía .bib 70

B.1. Tipos posibles de elementos en bibligrafía con natbib 72

LISTADOS

ABREVIATURAS

ESI	Escuela Superior de ingeniería
InSo	Ingeniería del Software
SBSE	Search based software engineering
TFG	Trabajo fin de grado
UAL	Universidad de Almería

RESUMEN Y ABSTRACT

Colivi nace como una plataforma web dirigida a solventar las trabas habituales al buscar piso compartido y organizar el día a día entre estudiantes. Se resolvieron dos problemas tradicionalmente desconectados: la captación de habitaciones y la convivencia interna.

El sistema se apoya en un backend desarrollado con Java 21 sobre Spring Boot 3 y persistencia en PostgreSQL, enlazado con un cliente web interactivo en React. A esta base técnica se añade un microservicio independiente en Node.js que implementa el protocolo MCP (*Model Context Protocol*). Gracias a esta pieza, el modelo de lenguaje consulta anuncios disponibles, inspecciona los turnos de limpieza pendientes y extrae resúmenes de la mensajería interna sin mutar datos en el servidor.

Las decisiones de diseño priorizan el desacoplamiento mediante principios SOLID y patrones de diseño guiado por el dominio (DDD). Se abordaron retos técnicos exigentes: la concurrencia en la reserva de habitaciones mediante bloqueos en base de datos, el cálculo determinista de balances contables entre compañeros y la gamificación en el reparto de tareas domésticas.

Colivi addresses the recurrent friction students face when searching for shared flats and organizing daily life with roommates. Finding rooms from abroad during exchange programs usually means dealing with fragmented listings and unverified landlords. Once settled, chores and shared expenses trigger avoidable tensions.

The architecture combines a Java 21 backend built on Spring Boot 3 with a PostgreSQL relational database and an interactive React web frontend. An external Node.js microservice exposes tools through the Model Context Protocol (MCP), enabling an LLM agent to inspect household chore rotations, query housing listings, and summarize inbox notifications safely under read-only boundaries.

Engineering decisions focused on SOLID principles and Domain-Driven Design (DDD). The implementation tackles strict database-level concurrency controls during booking transitions, graph-based debt balance resolutions between tenants, and a fair round-robin task assignment engine.

1 Introducción

El acceso a la vivienda compartida define la rutina de miles de universitarios y jóvenes profesionales en toda España. La escalada de precios del alquiler en las ciudades universitarias transformó una experiencia temporal de dos cursos en un régimen residencial obligado que se prolonga durante años. Alquilar una habitación ya no es una elección casual. Supone la única alternativa económica viable ante el encarecimiento desmedido de los inmuebles completos.

Los programas de intercambio internacional intensifican este problema. Cada semestre llegan estudiantes bajo convenios Erasmus que se ven forzados a reservar habitación a ciegas desde el extranjero, sin visitar el piso físico y negociando en idiomas que apenas dominan. Fiarse de fotografías sueltas en anuncios dudosos genera incertidumbre real. El riesgo económico resulta patente.

La dispersión de canales agrava estas dificultades iniciales. La mayoría de arrendamientos entre particulares se siguen tramitando mediante grupos informales en redes sociales, anuncios clasificados genéricos o acuerdos verbales sin respaldo. Esta falta de estructura acarrea abusos contractuales, pérdida de depósitos y un cruce caótico de mensajes entre las partes antes de cerrar cualquier trato.

Las fricciones continúan una vez dentro de la vivienda. Entrar al piso abre un segundo frente organizativo: quién limpia las zonas comunes, cómo se liquidan las facturas de suministros y qué normas rigen los espacios compartidos. Resolver estas responsabilidades con notas adhesivas o hilos caóticos de mensajería fracasa sistemáticamente. Se generan malentendidos y desequilibrios en el reparto doméstico que desgastan la convivencia diaria.

1.1 Justificación

El desarrollo de Colivi surge de una necesidad práctica vivida en primera persona durante estancias de intercambio universitario. Buscar habitación a cientos de kilómetros revela un mercado opaco. Los anuncios omiten detalles esenciales, los arrendadores tardan días en contestar y las condiciones económicas varían sobre la marcha sin dejar constancia escrita. El estudiante asume un desembolso económico sin garantías reales sobre el estado del piso ni la identidad de quien cobra la fianza.

La convivencia posterior destapa nuevos roces cotidianos. Una vez formalizada la estancia, el grupo debe coordinar turnos de limpieza, compras compartidas y pagos de suministros. Confiar este control a pizarras domésticas o chats grupales suele terminar en disputas. La

ausencia de un registro inmutable dificulta comprobar quién ha cumplido sus obligaciones y quién acumula retrasos continuados en los gastos.

Las aplicaciones comerciales resuelven únicamente fragmentos aislados de este ciclo. Existen portales inmobiliarios centrados en captar inquilinos y aplicaciones móviles diseñadas para anotar gastos entre amigos, pero ninguna herramienta enlaza la reserva formal con la vida diaria dentro de la vivienda. Colivi conecta ambos momentos bajo un mismo sistema. La plataforma combina un backend en Spring Boot 3 con tipado estricto, una interfaz reactiva construida en React y un servidor auxiliar bajo el protocolo MCP (*Model Context Protocol*) para consultar datos mediante modelos de lenguaje, cerrando la brecha entre la búsqueda inicial y el régimen diario del piso.

1.2 Objetivos y Estructura

Diseñar y construir Colivi constituye el objetivo principal de este proyecto, ofreciendo una plataforma web que da respuesta a la búsqueda de habitaciones y a la organización interna en pisos de estudiantes. La aplicación acompaña a los usuarios durante todo el ciclo: conecta al arrendatario con el propietario durante la reserva y estructura el reparto de tareas domésticas una vez comenzada la convivencia.

La consecución de esta meta global se desglosa en los siguientes objetivos específicos:

1. Implementar un módulo de seguridad para registro e inicio de sesión mediante tókenes JWT, distinguiendo perfiles de inquilino y propietario.
2. Habilitar la edición del perfil de usuario con datos de contacto, preferencias y hábitos de convivencia.
3. Desarrollar un buscador de habitaciones con filtrado por localización geográfica, precio mensual, equipamiento y normas de estancia.
4. Construir un canal de mensajería contextual asociado a cada anuncio que permita plantear dudas al anfitrión sin exigir una reserva previa.
5. Programar el ciclo de vida de las solicitudes de reserva con control estricto de estados y depósitos de fianza.
6. Diseñar un planificador gamificado de tareas domésticas basado en turnos rotativos (*Round-Robin*), tabla de puntuaciones y posibilidad de rescate entre compañeros.
7. Disponer un registro inmutable de auditoría para monitorizar acciones clave dentro del hogar, como la realización de limpiezas o la incorporación de nuevos miembros.
8. Integrar un asistente inteligente fundamentado en el protocolo *Model Context Protocol* (MCP) que consulte turnos, resuma actividad o filtre anuncios sin modificar la base de datos.

La arquitectura global del sistema (reflejada en la figura 1.1) distribuye sus responsabilidades en cuatro capas desacopladas: el cliente web, el servidor central de aplicaciones, el microservicio de soporte inteligente y el motor de persistencia relacional.

El cliente web interactúa directamente con el usuario mediante una interfaz reactiva construida con React, TypeScript y Tailwind CSS. Vite actúa como empaquetador para optimizar los tiempos de recarga durante el desarrollo y minimizar el tamaño del paquete final en producción. La sincronización de datos asíncronos recae en TanStack Query, aplicando una estrategia de sondeo controlado (*Smart Polling*) que refresca los mensajes y las tareas únicamente mientras la pestaña del navegador permanece en primer plano, evitando la sobrecarga de conexiones persistentes innecesarias.

El backend principal se ejecuta sobre la máquina virtual de Java 21 utilizando Spring Boot 3. Este servidor expone una API REST dividida en módulos de dominio bajo principios SOLID. Procesa reglas de negocio críticas: el reparto equitativo de turnos de limpieza, la desvinculación automática de reservas mediante eventos internos de dominio (@EventListener) y la contención de colisiones concurrentes mediante bloqueos pesimistas en las tablas afectadas. Asimismo, se apoya en Spring AI para consultar modelos de lenguaje remotos a través de la API de Groq, devolviendo respuestas formateadas en JSON estricto.

Un microservicio complementario desarrollado en Node.js y TypeScript expone un servidor MCP oficial mediante *Server-Sent Events* (SSE). Este servicio declara herramientas de inspección de anuncios, consulta de estado de tareas y cola de moderación. Cada conexión hereda el token JWT emitido por el backend, imponiendo un aislamiento estricto de solo lectura para evitar que la inteligencia artificial altere tablas de forma directa.

La persistencia de datos reside en un clúster relacional de PostgreSQL. Mediante JPA e Hibernate se gestionan las entidades de usuarios, publicaciones, conversaciones, trazas de auditoría y turnos periódicos. Se aplicaron índices compuestos y restricciones de unicidad para descartar registros duplicados y mantener la integridad referencial en todo momento.

1.3 PLANIFICACION

El proyecto abarca una carga docente estipulada de 12 ECTS, equivalente a 300 horas de dedicación en el plan de estudios del Grado en Informática. El cronograma cubre la toma inicial de requisitos, el modelado del dominio, la codificación del sistema distribuido, la integración de modelos de lenguaje y la redacción de la memoria. La figura 1.2 recoge la descomposición temporal de este trabajo.

El trabajo preliminar absorbió 100 horas repartidas durante el primer mes: evaluación tecnológica (30 horas), especificación de requisitos del sistema (30 horas) y diseño de maquetas de interfaz (40 horas). Esta fase sirvió para contrastar la viabilidad del protocolo MCP con Spring AI, verificar la velocidad de inferencia del proveedor Groq, normalizar las tablas en PostgreSQL y cerrar los flujos de navegación antes de codificar la solución.

El núcleo de programación concentró 170 horas de esfuerzo. La construcción de la aplicación web y los microservicios acaparó 110 horas, programando la lógica de negocio en Spring Boot 3, el cliente web en React con TanStack Query y el servidor MCP en Node.js. Sobre esta base funcional se desarrollaron tres hitos derivados: el empaquetado móvil sobre Android con Capacitor (25 horas), la definición de contenedores Docker y el despliegue en la nube de Microsoft Azure (20 horas), y una batería final de pruebas de integración con JUnit y Mockito (15 horas) para verificar la consistencia transaccional y la autenticación sin estado.

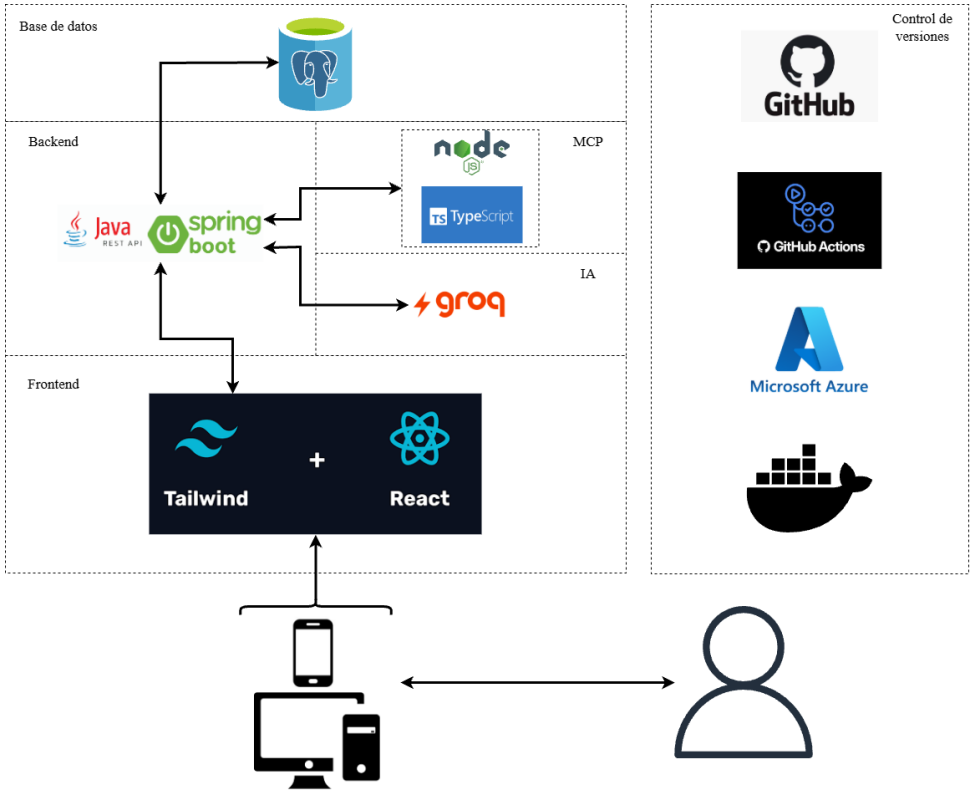


Figura 1.1: Estructura técnica global y arquitectura de la plataforma Colivi

Las 30 horas finales se asignaron a redactar la memoria en LaTeX conforme a las pautas tipográficas de la Escuela Superior de Ingeniería, generar los diagramas formales en PlantUML y preparar la defensa ante el tribunal. La redacción se abordó con el software ya estabilizado, garantizando que el texto reflejara el comportamiento exacto de las rutas, entidades y eventos implementados.

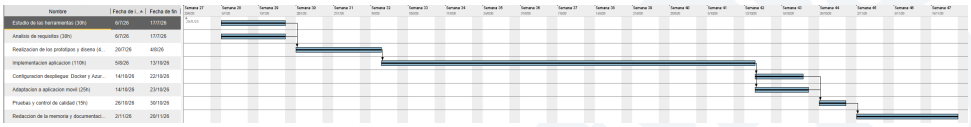


Figura 1.2: Planificación Temporal General

Las 110 horas dedicadas al desarrollo exigieron planificar qué módulos podían ejecutarse en paralelo y cuáles imponían una secuencia obligatoria por dependencias de datos.

El trabajo simultáneo aceleró las capas base: mientras se levantaban las entidades JPA y las migraciones en PostgreSQL, se maquetaba el armazón de rutas y componentes en React con Tailwind CSS. La autenticación local mediante tókenes JWT y el inicio de sesión federado con Google OAuth se construyeron en paralelo a sus formularios web en el cliente. De igual manera, el buscador de alojamientos avanzó al mismo tiempo que el mapa interactivo de Leaflet, pues ambas vistas consumían los mismos endpoints REST del backend.

Las reglas de negocio troncales requirieron un avance estrictamente secuencial debido a restricciones del modelo relacional:

1. **Gestión de alojamientos y catálogo**: habilitó la persistencia de propiedades, la subida de fotografías a Cloudinary y la definición de comodidades antes de admitir cualquier interacción.

2. **Solicitudes de reserva y cobro de fianza**: se apoyó en el catálogo activo para gestionar rangos de fechas sin colisiones temporales y conciliar el abono de la fianza.

3. **Espacio del hogar**: se instanció como contenedor lógico derivado de una reserva confirmada y abonada, agrupando a los inquilinos aceptados.

4. **Gastos compartidos y tareas domésticas**: dependían de la existencia previa del hogar. Se implementó el motor de saldos contables con simplificación de deudas y, tras verificarlo, se programó el reparto rotativo de limpieza con asignaciones periódicas y recompensas.

5. **Mensajería contextual**: se construyó sobre la convivencia constituida, permitiendo conversaciones entre compañeros de piso o entre inquilinos y propietarios.

6. **Asistente virtual (Spring AI y MCP)**: se integró tras estabilizar los endpoints de catálogo y tareas, permitiendo que el servidor MCP consumiera datos consolidados sin lidiar con contratos cambiantes en la API interna.

7. **Moderación, valoraciones y RGPD**: cerró el sistema antes de la fase de despliegue, añadiendo denuncias disciplinarias, reseñas mutuas y políticas de supresión de datos personales.

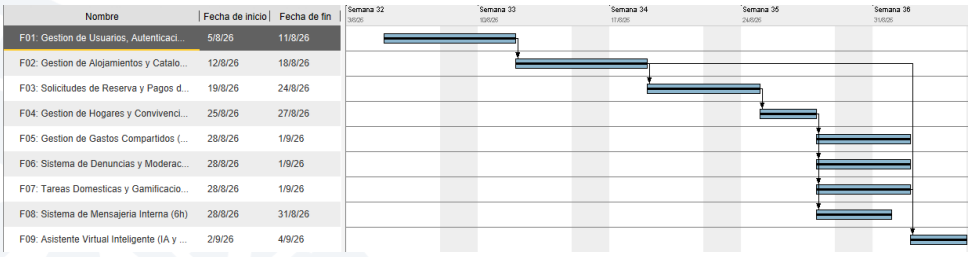


Figura 1.3: Planificación Temporal de la implementación

1.4 Apps Similares

Examinar el mercado de software actual permitió delimitar los patrones de diseño consolidados y constatar qué carencias sufren los usuarios en su día a día. Existen herramientas consolidadas para cada fase por separado, pero ninguna cubre el recorrido completo. Se seleccionaron cinco plataformas representativas: portales de captación inmobiliaria generalista como Idealista [1], servicios de alojamiento temporal como Airbnb [2], redes de alquiler por habitaciones como Badi [3], gestores contables como Tricount [4] y organizadores de tareas domésticas como Flatastic [5].

Idealista [1] (figura 1.4a) domina el mercado inmobiliario en España gracias a su catálogo masivo y a un motor de búsqueda geográfica ágil. Su modelo de negocio se limita a la

intermediación publicitaria inicial. El portal se desentiende por completo una vez establecido el contacto entre anunciante e interesado: no custodia fianzas, no verifica identidades mediante contratos formales y carece de herramientas para organizar la vida comunitaria tras la entrega de llaves.

Airbnb [2] (figura 1.4b) dispone de una pasarela de pagos consolidada, gestión de depósitos en garantía y verificación de identidades mediante valoraciones recíprocas. Sin embargo, su orientación es turística. Las comisiones por servicio resultan prohibitivas para alquileres residenciales de media y larga duración. Tampoco contempla dinámicas de convivencia entre compañeros que comparten un mismo piso durante meses, delegando cualquier acuerdo interno a la comunicación informal externa.

Badi [3] (figura 1.4c) nació orientada al alquiler de habitaciones en grandes núcleos urbanos, filtrando cuartos libres y perfiles de candidatos mediante un chat integrado. Pese a coincidir con la fase de captación de Colivi, su utilidad concluye con el acuerdo de entrada. La aplicación no acompaña a los inquilinos en el día a día: queda fuera cualquier herramienta para auditar gastos compartidos, coordinar la limpieza o dirimir desacuerdos domésticos.

Tricount [4] (figura 1.4d) resuelve con solvencia el reparto de gastos mediante la simplificación de grafos de deuda entre múltiples participantes. Funciona como una libreta contable aislada. Carece de contexto inmobiliario real: los compañeros deben introducir cada concepto a mano sin vincularlo a contratos de alquiler, recibos oficiales o depósitos de fianza gestionados de forma conjunta con el propietario.

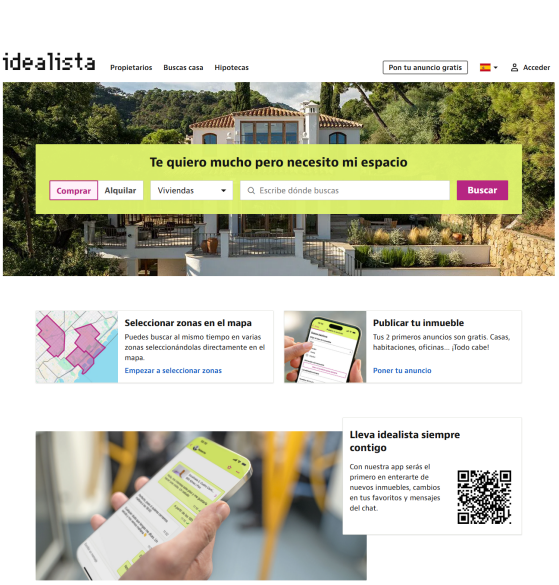
El ecosistema digital actual fragmenta la experiencia del inquilino en aplicaciones independientes sin conexión entre sí. Los usuarios buscan habitación en una web, firman el contrato en persona, anotan los gastos en una aplicación externa y discuten la limpieza por mensajería. Colivi unifica estas fases bajo una sola plataforma web y móvil, incorporando además un asistente basado en MCP para interactuar con los datos del hogar.

Tabla 1.1: Comparativa de características entre plataformas analizadas y Colivi

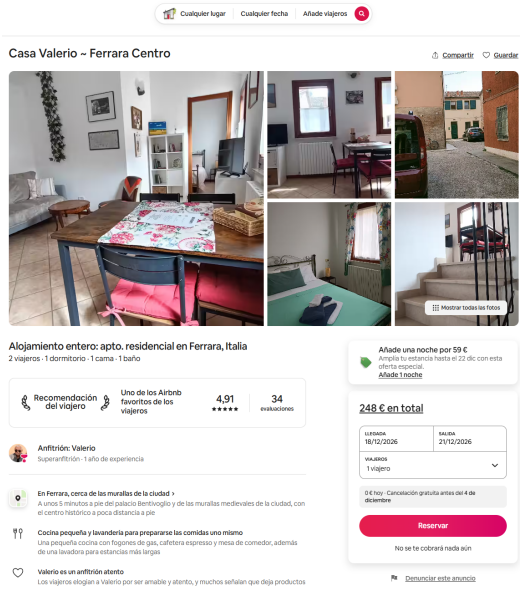
Plataforma	Catálogo y Búsqueda	Reserva y Fianza Integrada	Reparto de Gastos Comunes	Gamificación de Tareas	Asistente IA (MCP)	Enfoque Coliving
Idealista	Sí	No	No	No	No	No
Airbnb	Sí	Sí	No	No	No	No
Badi	Sí	No	No	No	No	Sí
Tricount	No	No	Sí	No	No	No
Colivi (Propuesta)	**Sí**	**Sí**	**Sí**	**Sí**	**Sí**	**Sí**

Las columnas de la tabla comparativa evalúan las siguientes capacidades técnicas y de dominio:

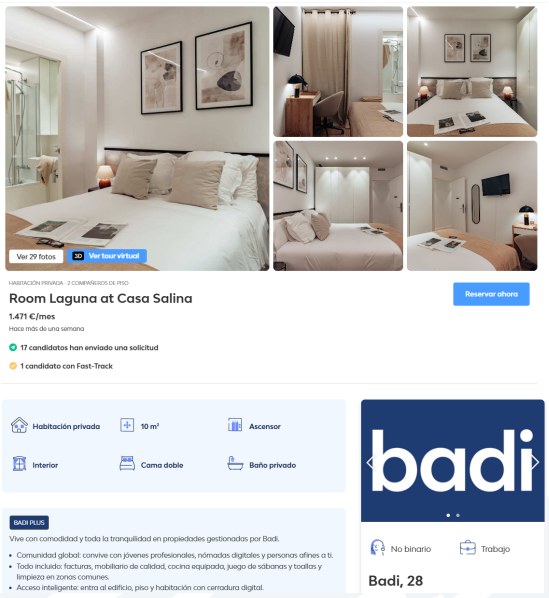
- **Catálogo y Búsqueda**: motor público de localización con filtros espaciales, rangos de precio y comodidades del inmueble. Portales como Idealista, Airbnb y Badi satisfacen este criterio, mientras que utilidades contables como Tricount o Flatastic carecen de catálogo por definición.
- **Reserva y Fianza Integrada**: capacidad para formalizar acuerdos de estancia y retener o transferir depósitos de garantía mediante pasarelas auditables dentro del propio siste-



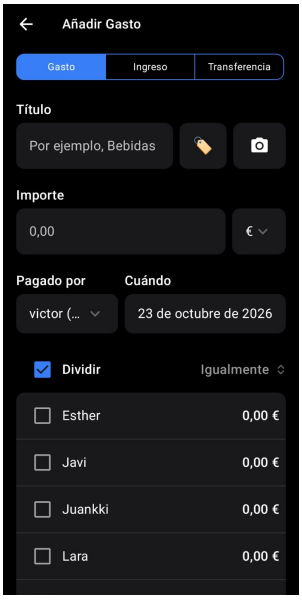
(a) Búsqueda en Idealista



(b) Detalles y reserva en Airbnb



(c) Habitaciones y perfiles en Badi



(d) Reparto de gastos en Tricount

Figura 1.4: Interfaces de las principales soluciones analizadas en el mercado

ma. Solo Airbnb soporta este flujo en el mercado actual, pues Idealista y Badi delegan las transacciones económicas a pactos privados fuera de la plataforma.

- **Reparto de Gastos Comunes**: presencia de un motor de liquidación para prorratear facturas, calcular saldos individuales y registrar transferencias compensatorias. Supone el núcleo de Tricount, pero los portales de alquiler lo omiten en su totalidad.
- **Gamificación de Tareas**: mecanismos de rotación periódica del trabajo doméstico apoyados en asignaciones equitativas y tablas de clasificación. Únicamente Flatastic aproxima este concepto, aunque con métricas básicas sin trazabilidad inmutable del hogar.
- **Asistente IA (MCP)**: integración de un agente conversacional capaz de ejecutar herramientas sobre la base de datos del piso mediante el protocolo *Model Context Protocol*. Ningún competidor del mercado incorpora agentes con acceso contextual al estado de tareas o finanzas de la vivienda.
- **Enfoque Coliving**: diseño centrado en el arrendamiento por habitaciones con zonas comunes y convivencia comunitaria entre estudiantes o jóvenes profesionales. Badi y Flatastic comparten esta orientación, diferenciándose de modelos turísticos o residenciales familiares.

Determinadas características exigen matices técnicos. Badi, por ejemplo, gestiona un cobro inicial para formalizar la reserva, pero no asume la custodia ni la restitución de la fianza durante la vigencia del contrato de arrendamiento. De igual forma, las opciones de atención automatizada en portales como Airbnb se limitan a centros de ayuda estáticos con preguntas frecuentes, sin conectividad hacia un servidor de contexto capaz de inspeccionar los registros internos del alojamiento.

1.5 Contenido de la memoria

El documento organiza el trabajo de ingeniería a través de los siguientes capítulos y apartados temáticos:

- **Introducción y Justificación**: planteamiento del problema en la búsqueda de alojamiento universitario, análisis de la demanda y exposición de los roces organizativos en pisos compartidos.
- **Objetivos y Planificación**: delimitación del alcance funcional del sistema web y móvil, acompañada del cronograma de 300 horas lectivas y la secuenciación de dependencias entre componentes.
- **Aplicaciones similares**: estudio de mercado sobre soluciones inmobiliarias y contables (Idealista, Airbnb, Badi, Tricount y Flatastic), contrastando sus limitaciones frente a una plataforma integral.
- **Herramientas y metodologías**: justificación del desarrollo ágil con TDD y revisión del ecosistema técnico: Java 21, Spring Boot 3, PostgreSQL, React, TanStack Query, el servidor MCP en Node.js y el empaquetado con Capacitor.

- **Análisis funcional**: definición de actores, catálogo de requisitos funcionales y especificación formal de casos de uso mediante diagramas UML y tablas estructuradas.
- **Diseño y arquitectura**: modelo entidad-relación en base de datos, descomposición en capas bajo principios SOLID, diagramas de secuencia para flujos transaccionales y diseño de interfaz en Figma.
- **Implementación**: desarrollo modular del backend y cliente, cubriendo autenticación JWT, buscador con mapas, reservas con fianza, motor de gastos, turnos de limpieza y microservicio de soporte inteligente.
- **Adaptación móvil y despliegue**: configuración del contenedor Docker, compilación del APK para Android mediante Capacitor y despliegue en Microsoft Azure.
- **Pruebas y calidad**: baterías de tests unitarios e integrados con JUnit 5 y Mockito, validación de endpoints y resolución de problemas de concurrencia.
- **Estudio de costes**: balance económico del proyecto computando dedicación horaria, licencias de software y consumo de infraestructura en la nube.
- **Conclusiones y líneas futuras**: balance del cumplimiento de objetivos y propuestas técnicas para incorporar pagos bancarios directos y calendarios externos.
- **Bibliografía**: catálogo de estándares, libros de ingeniería del software, documentación técnica oficial y referencias académicas.

2 Herramientas y metodologías

Construir una arquitectura distribuida de forma individual exige combinar rigor en el código con agilidad en las entregas. Se adoptó una metodología ágil adaptada, reforzada con la práctica de desarrollo guiado por pruebas o TDD (*Test-Driven Development*). Esta disciplina permitió dividir el desarrollo en ciclos cortos de especificación, codificación y verificación inmediata. Cada módulo quedaba validado antes de enlazarlo con el resto del sistema.

TDD blindó las partes más sensibles del dominio: el cálculo de saldos en gastos compartidos y la concurrencia en la reserva de habitaciones. Redactar las pruebas con anterioridad fijó el comportamiento esperado de servicios y repositorios sin ambigüedades. Durante este proceso, se emplearon modelos de lenguaje como asistentes de apoyo para explorar casos límite en las aserciones y sugerir refactorizaciones orientadas a principios SOLID (*Single responsibility, Open-closed, Liskov substitution, Interface segregation, Dependency inversion*).

El stack técnico se seleccionó priorizando tipado estático, rendimiento transaccional y bajo acoplamiento:

- **Entorno de desarrollo**: Antigravity centralizó el espacio de trabajo coordinando la edición simultánea del backend y los microservicios, terminales concurrentes y automatización de tareas.
- **Lógica de negocio y servicios**: el backend principal se programó en Java 21 sobre Spring Boot 3, integrando Spring AI para la comunicación con LLMs y la ejecución de llamadas a herramientas. Paralelamente, se construyó un servidor MCP (*Model Context Protocol*) en Node.js y TypeScript que expone sus endpoints mediante *Server-Sent Events* (SSE).
- **Interfaz web y empaquetado móvil**: el cliente web se programó en React con TypeScript y empaquetado con Vite. Tailwind CSS resolvió el diseño visual y TanStack Query gestionó las consultas en segundo plano. Mediante Capacitor se generó el paquete nativo instalable en Android reutilizando el código web.
- **Almacenamiento de datos**: PostgreSQL administra las transacciones relacionales, consumidas desde Java mediante JPA e Hibernate con bloqueos explícitos para evitar conflictos de concurrencia.
- **Prototipado y diagramación**: las pantallas se diseñaron en Figma para escritorio y móvil. PlantUML generó los diagramas UML directamente desde ficheros de texto plano, mientras draw.io sirvió para los esquemas arquitectónicos.

- **Control de versiones y cronograma**: GitHub alojó los repositorios bajo un flujo estricto de ramas y solicitudes de extracción (*pull requests*). El seguimiento de tiempos y fechas límite se controló en GanttProject.

Los siguientes apartados profundizan en la disciplina metodológica elegida y en la función de cada herramienta dentro de la arquitectura del proyecto.

2.1 Metodologías usadas

La guía SWEBOK (*Software Engineering Body of Knowledge*) [6] divide la ingeniería del software en procesos disciplinados para planificar, ejecutar y controlar el desarrollo frente a restricciones reales. Elegir un método de trabajo determina cómo evoluciona el código y cómo se responde a cambios en los requisitos sin comprometer la estabilidad del sistema.

2.1.1 Revisión de enfoques metodológicos

El modelo tradicional en cascada exige completar y congelar cada fase antes de iniciar la siguiente: requisitos, diseño, codificación, verificación y entrega. Si bien funciona en proyectos con especificaciones cerradas y contratos rígidos, fracasa al integrar servicios desacoplados o protocolos emergentes como MCP. Descubrir un fallo de integración en la fase final de pruebas dispararía el coste de rehacer módulos enteros.

Scrum estructura el trabajo en iteraciones cortas llamadas *sprints*, orientadas a entregar incrementos funcionales cada dos o cuatro semanas. Reparte responsabilidades entre perfiles definidos (*Scrum Master*, *Product Owner*, equipo de desarrollo) y exige reuniones periódicas de sincronización y retrospectiva. Pese a dominar en la industria corporativa, su burocracia de reuniones pierde sentido en un proyecto individual sin equipo que coordinar. La figura 2.1 refleja la comparativa histórica de tasas de éxito entre ambos enfoques.

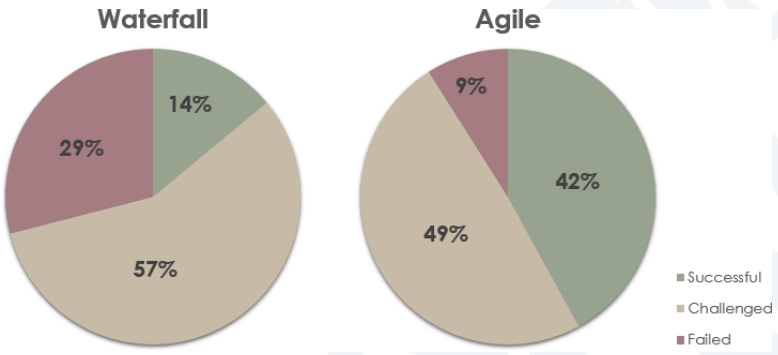


Figura 2.1: Diferencia de éxito entre metodologías ágiles y tradicionales en cascada

Kanban elimina las cadencias temporales fijas y se centra en limitar el trabajo en curso o WIP (*Work In Progress*). Las tareas avanzan a través de columnas de estado en un tablero visual (figura 2.2): pendientes, en desarrollo y completadas. Este flujo continuo permite priorizar el trabajo según cuellos de botella reales, facilitando al programador concentrarse en un solo componente hasta cerrarlo.

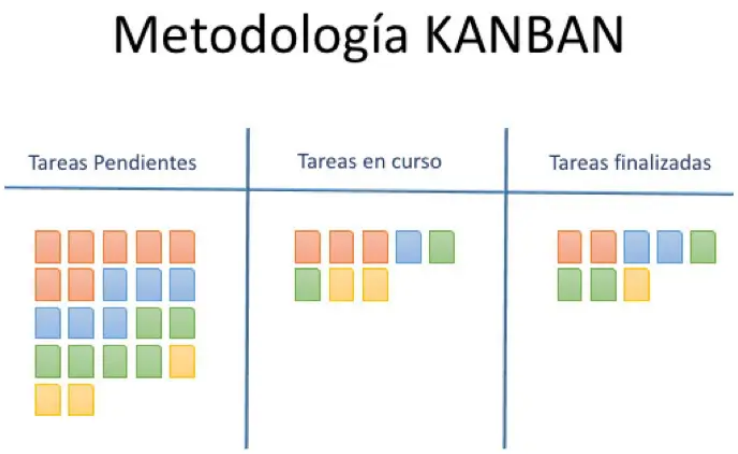


Figura 2.2: Tablero Kanban representativo del flujo de desarrollo ágil

La programación extrema (*Extreme Programming* o XP) antepone la excelencia en el código a la gestión burocrática. Sus pilares incluyen la refactorización continua, la integración frecuente y el desarrollo guiado por pruebas o TDD (*Test-Driven Development*). En TDD, los tests se redactan antes que el código de producción, garantizando que cada función responda a una necesidad validada y medible.

2.1.2 Análisis comparativo y selección para el proyecto

La arquitectura de Colivi —un backend modular en Spring Boot 3, un servidor MCP independiente en Node.js, un cliente web en React y el paquete móvil con Capacitor— hacía inviable una cascada estricta. Conectar herramientas de inteligencia artificial con inferencia remota exigía experimentar con esquemas JSON y ajustar contratos sobre la marcha.

Scrum tampoco encajaba en un escenario unipersonal sin ceremonias diarias. Por estas razones, se diseñó una estrategia híbrida adaptada a las necesidades reales del desarrollo:

- **Gestión del flujo de trabajo**: Kanban organizó el avance técnico mediante un tablero en GitHub Projects, vinculando incidencias, dependencias y confirmaciones de código sin imponer cortes temporales artificiales.
- **Disciplina técnica con TDD**: las prácticas de XP guiaron la implementación mediante el ciclo de diseño, prueba y refactorización resumido en la figura 2.3. Este mecanismo protegió el cálculo financiero de deudas y la reserva concurrente de habitaciones, forzando un diseño desacoplado bajo principios SOLID (*Single responsibility, Open-closed, Liskov substitution, Interface segregation, Dependency inversion*).
- **Integración asistida**: el entorno Antigravity aceleró la formulación de baterías de pruebas con JUnit y Mockito, complementando la inspección manual con compilaciones automatizadas en ramas de características (*feature branches*).

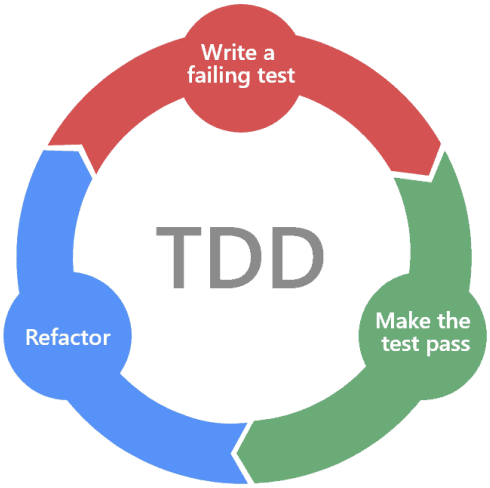


Figura 2.3: Ciclo iterativo de la metodología TDD (*Test-Driven Development*)

2.2 Herramientas usadas

Cada herramienta del proyecto responde a una exigencia técnica concreta de la arquitectura distribuida. Se descartaron soluciones monolíticas pesadas en favor de tecnologías con tipado estricto, bajo consumo de memoria y separación limpia entre dominios.

2.2.1 Entornos de desarrollo y asistencia técnica

Antigravity actuó como entorno principal para programar el backend en Spring Boot y el microservicio en Node.js (figura 2.4). Su soporte para espacios de trabajo desacoplados facilitó mantener abiertos los servicios del servidor mientras se levantaban contenedores y terminales de depuración simultáneas. Frente a alternativas lentas como Eclipse, agilizó la navegación entre clases de dominio y la refactorización guiada.

Visual Studio Code asumió el desarrollo del cliente web y el empaquetado móvil. El ecosistema de complementos para TypeScript, la integración con ESLint y el formateo automático con Prettier garantizaron coherencia estilística en todo el código de React. Alternativas como WebStorm ofrecían prestaciones equivalentes pero con una penalización innecesaria en el consumo de memoria RAM del equipo de desarrollo.

Android Studio intervino en la fase final de empaquetado y prueba sobre dispositivos móviles. La lógica de la aplicación se comparte desde React gracias a Capacitor; sin embargo, compilar el paquete nativo APK (*Android Package Kit*), firmar digitalmente el binario y depurar las llamadas al puente de WebView requirió la suite oficial de Android sobre emuladores del sistema operativo.

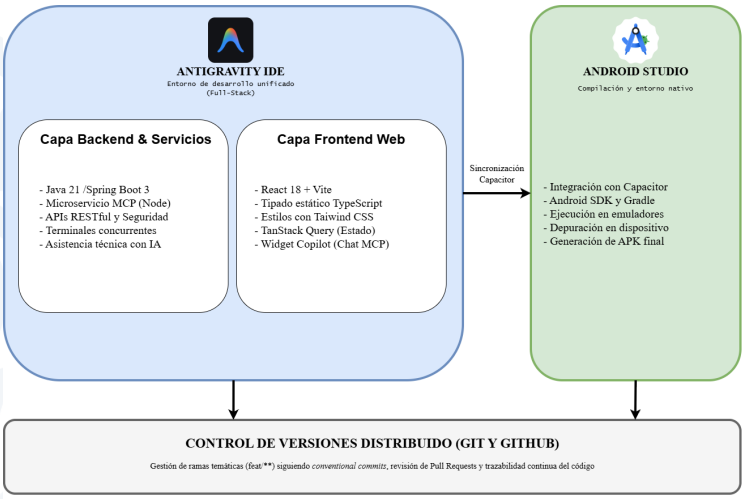


Figura 2.4: Entornos de desarrollo utilizados en el proyecto. Elaboración propia

2.2.2 Lenguajes, librerías y servicios

Java 21 y Spring Boot 3 forman el núcleo del servidor. Los registros inmutables (*records*) y la concordancia de patrones (*pattern matching*) de Java 21 aligeraron los objetos de transferencia DTO (*Data Transfer Object*) eliminando código repetitivo. Spring Boot 3 resolvió la inyección de dependencias, las transacciones declarativas (@Transactional) y la seguridad

sin estado con Spring Security mediante filtros para tókenes JWT (*JSON Web Tokens*). Frente a soluciones basadas en Python o JavaScript, el tipado de Java evitó errores de redondeo en los cálculos contables. Spring AI facilitó además enlazar este backend con los modelos de lenguaje servidos por la API de Groq, forzando respuestas estructuradas en esquemas JSON.

El microservicio del asistente inteligente se desarrolló con Node.js y TypeScript bajo el estándar *Model Context Protocol* (MCP). Se aisló este módulo del backend central para que las herramientas de consulta operaran de forma independiente mediante *Server-Sent Events* (SSE). TypeScript garantizó que los esquemas de entrada y salida de las funciones coincidieran exactamente con los tipos declarados por el cliente.

React 18 estructura la interfaz web apoyándose en Vite como empaquetador. La compilación basada en módulos nativos (*ES Modules*) de Vite redujo drásticamente el tiempo de refresco en caliente (*Hot Module Replacement*). Tailwind CSS aceleró el diseño maquetando directamente en los componentes sin lidiar con hojas de estilo dispersas. TanStack Query centralizó la sincronización con el servidor: resuelve la caché, invalida consultas al mutar datos y refresca los saldos en segundo plano. Para la versión móvil se empleó Capacitor, envolviendo la base web en una vista nativa de Android sin reescribir la lógica en Kotlin.

2.2.3 Almacenamiento y persistencia de datos

PostgreSQL 16 gestiona la persistencia de datos (figura 2.5). La naturaleza financiera de Colivi —balances de gastos, retención de fianzas y reservas de habitaciones— exige cumplir de forma estricta las garantías ACID (*Atomicity, Consistency, Isolation, Durability*). Bases de datos no relacionales como MongoDB se descartaron por su incapacidad para garantizar transacciones multipartitas fiables sin configuraciones complejas. La comunicación entre las entidades Java y las tablas SQL se resolvió mediante JPA e Hibernate, optimizando las consultas mediante proyecciones DTO y fetch joins para atajar el problema de rendimiento por consultas $N + 1$.



Figura 2.5: Sistema gestor de bases de datos PostgreSQL

2.2.4 Control de versiones, infraestructura y utilidades auxiliares

Git y GitHub gestionaron el versionado del código fuente mediante un flujo riguroso de ramas y solicitudes de extracción (*pull requests*). Ningún cambio se incorporó a la rama principal sin revisión previa de pruebas unitarias.

Docker y Docker Compose estandarizaron el entorno de ejecución. Se configuraron contenedores para levantar en local la base de datos PostgreSQL, el servidor Spring Boot y el microservicio MCP, replicando la misma infraestructura que se despliega en producción sobre Microsoft Azure.

Figma (figura 2.6a) sirvió para definir la tipografía, paleta de colores y maquetas previas de las pantallas para móvil y escritorio. PlantUML (figura 2.6b) generó los diagramas UML directamente desde código de texto versionado en Git, mientras que draw.io facilitó la exportación de esquemas de bloques. Las pruebas manuales de endpoints REST y eventos SSE se ejecutaron con Postman (figura 2.6c) y cURL antes de escribir los tests automatizados con JUnit 5 y Mockito.



(a) Figma **(b)** PlantUML **(c)** Postman

Figura 2.6: Herramientas auxiliares empleadas para diseño de interfaz, modelado y verificación de API

3 Análisis funcional

El análisis funcional traduce las dinámicas del coliving en especificaciones de ingeniería rigurosas, delimitando la frontera del sistema antes de entrar en la codificación. Se modelan los contratos de interacción, las entidades de datos y las restricciones de seguridad que rigen la plataforma Colivi.

Los usuarios se clasifican según sus privilegios y su grado de participación en la convivencia, distinguiendo desde visitantes anónimos hasta administradores de plataforma. Los diagramas de casos de uso estructuran visualmente los flujos entre el navegador web y las APIs del servidor, agrupando las funcionalidades en paquetes de dominio.

La especificación técnica se completa con el catálogo formal de requisitos funcionales y no funcionales, fijando las restricciones de latencia, seguridad sin estado mediante JWT y transacciones ACID en PostgreSQL. Sobre esta base analítica se asientan el modelo relacional y las pantallas diseñadas en Figma para guiar la maquetación.

3.1 Diagramas de casos de uso

Los diagramas de casos de uso bajo el estándar UML 2.5 formalizan los límites del sistema, identificando quién ejecuta cada acción y qué subsistemas intervienen. La herramienta PlantUML permitió generar estas representaciones a partir de código fuente.

Reunir ochenta y seis casos de uso en un único diagrama saturaría la página A4 y destruiría la legibilidad del documento. Por este motivo, el sistema se divide en nueve paquetes temáticos. Antes de descender al detalle de cada paquete, resulta necesario definir los actores que intervienen en la aplicación.

3.1.1 Actores del sistema

El modelo contempla usuarios humanos (con roles que varían según el contexto de la vivienda) y servicios externos integrados mediante APIs seguras. La figura 3.1 representa las relaciones de generalización y herencia entre ellos.

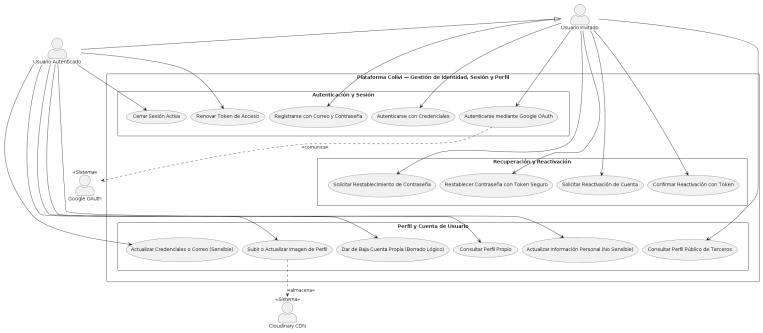


Figura 3.1: Jerarquía y relaciones de herencia entre los actores del sistema. Elaboración propia

Cada actor asume un rol delimitado en la aplicación:

- **Usuario Invitado**: navega de forma anónima sin sesión abierta. Explora el mapa de inmuebles, consulta fichas de habitaciones y solicita el registro o inicio de sesión.
- **Usuario Autenticado**: dispone de un token JWT válido. Gestiona sus datos de perfil, abre conversaciones en el chat, envía denuncias a moderación y formula preguntas al asistente virtual.
- **Inquilino**: usuario autenticado que busca piso. Emite solicitudes de reserva acotadas en el calendario, abona depósitos de fianza y puntúa las estancias finalizadas.
- **Anfitrión**: usuario autenticado que oferta propiedades. Da de alta inmuebles, publica anuncios con sus características físicas, administra la galería de fotos y acepta o declina solicitudes de reserva.
- **Compañero de Piso**: miembro integrado en un piso activo. Registra gastos comunes, revisa balances de deuda, transfiere liquidaciones y marca tareas de limpieza como completadas.
- **Administrador del Hogar**: compañero de piso responsable de coordinar la vivienda. Genera invitaciones efímeras, expulsa a residentes sin deudas pendientes y programa las tareas recurrentes.
- **Administrador del Sistema**: superusuario con permisos globales de moderación. Examina la cola de denuncias, audita conversaciones notificadas y bloquea cuentas o anuncios fraudulentos.
- **Google OAuth**: proveedor de identidad que expide credenciales federadas para agilizar el acceso sin contraseña local.
- **Cloudinary CDN**: infraestructura externa encargada de almacenar, transformar y servir imágenes de anuncios y avatares de usuario.
- **OpenStreetMap** / **Nominatim**: motor cartográfico consumido por el cliente web para transformar direcciones postales en coordenadas geográficas de latitud y longitud.

3.1.2 Especificación detallada de casos de uso

Cada caso de uso se especifica mediante una ficha estructurada que define su código identificador, el actor detonante, las precondiciones del dominio, la descripción funcional y el flujo secuencial de interacción entre el usuario y la plataforma.

3.1.3 Paquete 1: Gestión de identidad, sesión y perfil

El paquete centraliza el registro de cuentas, la expedición y rotación de tókenes de seguridad, el restablecimiento de contraseñas y el ciclo de vida del perfil. La figura 3.2 representa las relaciones entre los actores y los quince casos de uso del módulo.

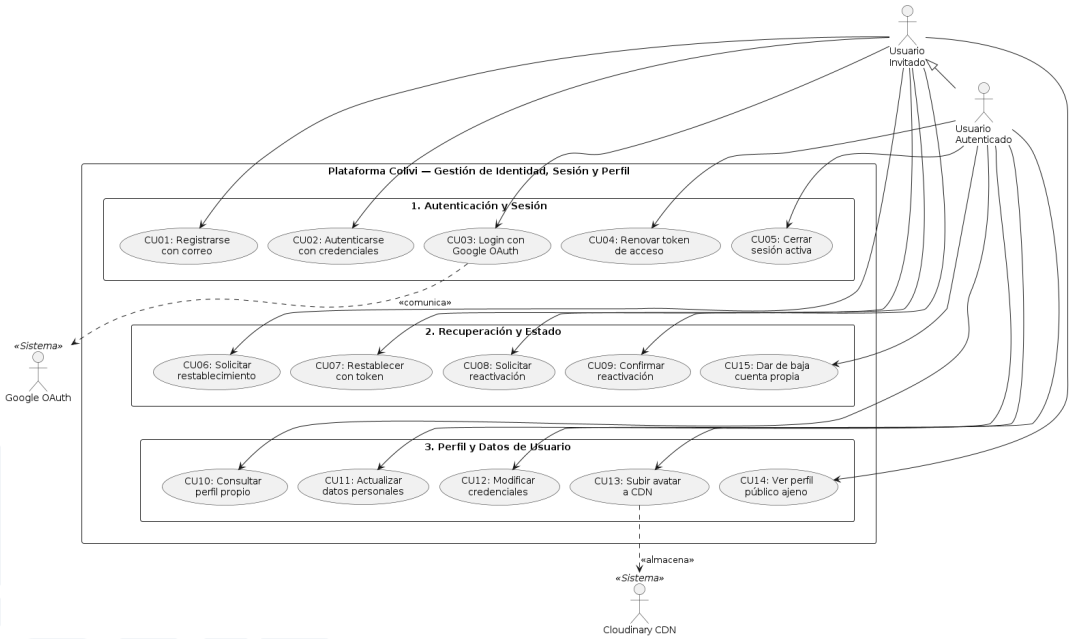


Figura 3.2: Diagrama de casos de uso del módulo de identidad, sesión y perfil

Las siguientes tablas recogen la especificación formal de cada caso de uso:

Caso de Uso	**CU01: Registrarse con correo y contraseña**
Actores	Usuario Invitado
Descripción	Permite a un nuevo usuario crear una cuenta local mediante credenciales
Precondiciones	No poseer una sesión activa y que el correo no figure previamente registrado
Flujo básico	1. El usuario accede al formulario de registro en la interfaz de usuario. 2. El usuario introduce nombre, apellidos, dirección de correo electrónico y contraseña. 3. El sistema valida el formato de los datos y comprueba la disponibilidad del correo. 4. El sistema almacena la credencial protegida con un algoritmo de dispersión criptográfica y confirma la creación de la cuenta.

Tabla 3.1: Especificación del caso de uso CU01.

Caso de Uso	**CU02: Autenticarse con credenciales**
Actores	Usuario Invitado
Descripción	Valida las credenciales locales para expedir una sesión sin estado
Precondiciones	Estar registrado y que la cuenta no se encuentre dada de baja o suspendida
Flujo básico	1. El usuario accede a la vista de inicio de sesión e introduce correo y contraseña. 2. El sistema valida la coincidencia de las credenciales frente a la base de datos. 3. El sistema genera un par de tókenes JWT (acceso y renovación) con los roles asignados. 4. El cliente almacena los tókenes de manera segura y redirige al panel principal.

Tabla 3.2: Especificación del caso de uso CU02.

Caso de Uso	CU03: Autenticarse mediante Google OAuth
Actores	Usuario Invitado, Google OAuth
Descripción	Permite iniciar sesión o registrarse mediante delegación de identidad federada
Precondiciones	Poseer una cuenta activa y verificada en el proveedor Google
Flujo básico	1. El usuario pulsa sobre la opción de acceso con Google en la interfaz. 2. El proveedor solicita el consentimiento y devuelve un token de identidad criptográfico. 3. El backend verifica la autenticidad del token frente a los servidores de Google. 4. El sistema localiza la cuenta existente o crea un nuevo registro sincronizado, emitiendo la sesión JWT local.

Tabla 3.3: Especificación del caso de uso CU03.

Caso de Uso	CU04: Renovar token de acceso
Actores	Usuario Autenticado
Descripción	Obtiene un nuevo token de acceso antes o tras la expiración del vigente
Precondiciones	Disponer de un token de renovación válido y no revocado
Flujo básico	1. El cliente detecta la caducidad del token de acceso durante una petición a la API. 2. El cliente envía el token de renovación a la ruta de refresco del servidor. 3. El backend comprueba la integridad del token y la vigencia del usuario. 4. El sistema expide y devuelve un nuevo token de acceso de corta duración.

Tabla 3.4: Especificación del caso de uso CU04.

Caso de Uso	CU05: Cerrar sesión activa
Actores	Usuario Autenticado
Descripción	Finaliza la sesión actual y purga las credenciales del cliente
Precondiciones	Mantener una sesión activa en la aplicación
Flujo básico	1. El usuario pulsa la acción de cierre de sesión en el menú de navegación. 2. El cliente elimina los tókenes de acceso y renovación almacenados localmente. 3. El sistema invalida la caché de consultas asíncronas y redirige a la vista de inicio.

Tabla 3.5: Especificación del caso de uso CU05.

Caso de Uso	**CU06: Solicitar restablecimiento de contraseña**
Actores	Usuario Invitado
Descripción	Inicia el protocolo de recuperación de acceso mediante correo electrónico
Precondiciones	Que la cuenta asociada al correo exista en la plataforma
Flujo básico	1. El usuario introduce su dirección de correo en la vista de contraseña olvidada. 2. El sistema comprueba la existencia del registro sin desvelar información sensible. 3. El sistema genera un token de un solo uso con caducidad temporal de quince minutos. 4. El servicio de correo despacha un mensaje con el enlace firmado para el restablecimiento.

Tabla 3.6: Especificación del caso de uso CU06.

Caso de Uso	**CU07: Restablecer contraseña con token seguro**
Actores	Usuario Invitado
Descripción	Actualiza la contraseña de la cuenta consumiendo el enlace de recuperación
Precondiciones	Acceder con un token de restablecimiento vigente y no consumido
Flujo básico	1. El usuario abre el enlace recibido e introduce una nueva contraseña. 2. El sistema valida las políticas de complejidad de la nueva clave. 3. El backend verifica la validez temporal del token y actualiza la credencial cifrada. 4. El sistema revoca el token utilizado y redirige a la vista de inicio de sesión.

Tabla 3.7: Especificación del caso de uso CU07.

Caso de Uso	CU08: Solicitar reactivación de cuenta
Actores	Usuario Invitado
Descripción	Permite a un usuario con la cuenta desactivada voluntariamente solicitar el enlace de reactivación
Precondiciones	Disponer de una cuenta registrada previamente en estado desactivado
Flujo básico	1. El usuario introduce su dirección de correo electrónico en el formulario de reactivación. 2. El sistema comprueba que la cuenta existe y se encuentra en estado inactivo. 3. El sistema genera un token criptográfico de reactivación con validez temporal. 4. El servicio de correo despacha un mensaje con el enlace firmado para confirmar el restablecimiento de la cuenta.

Tabla 3.8: Especificación del caso de uso CU08.

Caso de Uso	CU09: Confirmar reactivación de cuenta
Actores	Usuario Invitado
Descripción	Restablece el estado activo de la cuenta consumiendo el enlace de verificación recibido
Precondiciones	Acceder mediante un token de reactivación válido y no expirado
Flujo básico	1. El usuario accede a la plataforma a través del enlace de reactivación. 2. El backend valida la firma criptográfica y la vigencia temporal del token. 3. El sistema actualiza el estado de la cuenta a activo en la base de datos. 4. El sistema redirige al usuario al formulario de autenticación notificando el éxito del proceso.

Tabla 3.9: Especificación del caso de uso CU09.

Caso de Uso	CU10: Consultar perfil propio
Actores	Usuario Autenticado
Descripción	Muestra la información personal, preferencias de convivencia, valoraciones y estado de verificación del usuario actual
Precondiciones	Mantener una sesión activa en la aplicación
Flujo básico	1. El usuario accede a la sección de perfil desde la barra de navegación. 2. El sistema recupera los datos del usuario, métricas de actividad y configuración asociada. 3. La interfaz renderiza la información personal organizada en secciones editables.

Tabla 3.10: Especificación del caso de uso CU10.

Caso de Uso	**CU11: Actualizar datos personales**
Actores	Usuario Autenticado
Descripción	Permite modificar el nombre, teléfono, biografía académica o profesional y preferencias de convivencia
Precondiciones	Sesión activa y datos introducidos conformes a las restricciones de validación
Flujo básico	1. El usuario edita los campos deseados en el formulario de perfil. 2. El cliente valida el formato de los datos introducidos. 3. El backend persiste las modificaciones en la base de datos. 4. La interfaz actualiza la vista reflejando los nuevos datos confirmados.

Tabla 3.11: Especificación del caso de uso CU11.

Caso de Uso	**CU12: Modificar credenciales**
Actores	Usuario Autenticado
Descripción	Permite sustituir la contraseña de acceso actual por una nueva clave segura
Precondiciones	Estar autenticado con credenciales locales y conocer la contraseña vigente
Flujo básico	1. El usuario introduce la contraseña actual y la nueva clave junto con su confirmación. 2. El sistema verifica que la contraseña actual coincida con el resumen criptográfico almacenado. 3. El sistema comprueba las directivas de seguridad y complejidad de la nueva clave. 4. El backend genera el nuevo hash criptográfico, actualiza el registro y revoca las sesiones previas.

Tabla 3.12: Especificación del caso de uso CU12.

Caso de Uso	CU13: Subir avatar a CDN
Actores	Usuario Autenticado, Cloudinary CDN
Descripción	Permite cargar o sustituir la fotografía de perfil del usuario mediante almacenamiento en la nube
Precondiciones	Archivo gráfico en formato admitido inferior al límite de tamaño permitido
Flujo básico	1. El usuario selecciona una imagen desde el almacenamiento local de su dispositivo. 2. El cliente transfiere de forma segura el archivo multimedia al servicio Cloudinary. 3. El CDN procesa, optimiza y almacena la imagen, retornando su identificador y enlace seguro HTTPS. 4. El backend asocia la nueva URL al perfil del usuario y elimina el recurso anterior si procede.

Tabla 3.13: Especificación del caso de uso CU13.

Caso de Uso	CU14: Ver perfil público ajeno
Actores	Usuario Autenticado
Descripción	Permite inspeccionar la ficha pública de un potencial compañero de piso o anfitrión, omitiendo información de contacto privada
Precondiciones	Que el perfil consultado exista y pertenezca a un usuario con cuenta activa
Flujo básico	1. El usuario pulsa sobre el avatar o nombre de un usuario en un anuncio, mensaje o reseña. 2. El backend filtra los campos sensibles y extrae únicamente la información de visibilidad pública. 3. La interfaz muestra la biografía, estilo de vida, valoraciones recibidas y estado de verificación.

Tabla 3.14: Especificación del caso de uso CU14.

Caso de Uso	**CU15: Dar de baja cuenta propia**
Actores	Usuario Autenticado
Descripción	Permite al usuario solicitar la desactivación o supresión de su cuenta y sus datos personales de la plataforma
Precondiciones	No mantener contratos de alquiler vigentes, deudas pendientes ni reservas activas
Flujo básico	1. El usuario selecciona la opción de baja en los ajustes de seguridad e introduce su contraseña de confirmación. 2. El sistema audita la ausencia de bloqueos operativos, tales como estancias en curso o deudas en convivencias activas. 3. El backend marca la cuenta como inactiva o anonimiza los datos según la política de privacidad. 4. El sistema cierra la sesión activa, revoca los tókenes emitidos y redirige a la vista inicial.

Tabla 3.15: Especificación del caso de uso CU15.

3.1.4 Paquete 2: Gestión de inmuebles, publicaciones y confianza

Las funcionalidades de este paquete cubren la búsqueda de alojamientos, las valoraciones entre usuarios y la administración de propiedades por parte de los propietarios. Para mantener una escala legible en los diagramas UML, el contenido se estructura en dos bloques: la búsqueda pública con reseñas, y la gestión física de inmuebles y anuncios.

Exploración de alojamientos, recomendaciones y sistema de confianza

El primer bloque define la localización de habitaciones, la visualización en mapas y el intercambio de reseñas entre residentes. La figura 3.3 muestra las relaciones entre actores y los ocho casos de uso del módulo (CU16 a CU23).

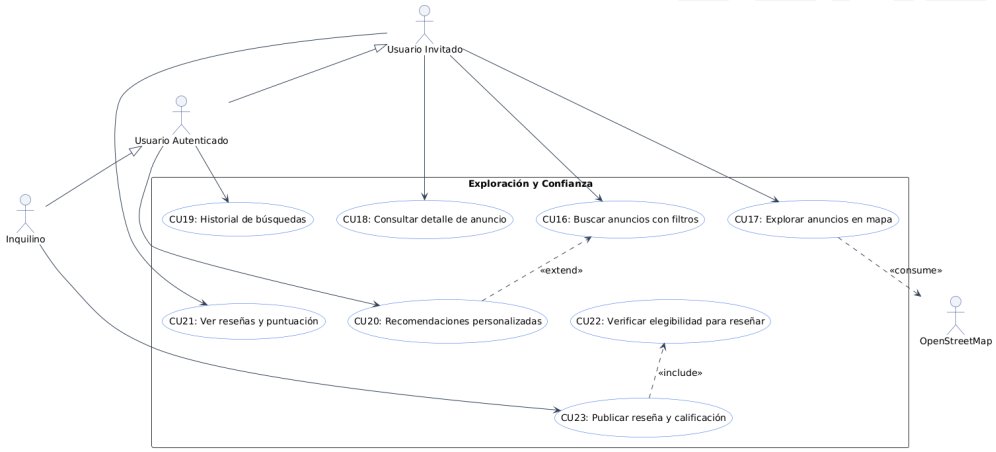


Figura 3.3: Diagrama de casos de uso de exploración, recomendaciones y reseñas

Las fichas de especificación funcional para este bloque comprenden desde CU16 hasta CU23:

Caso de Uso	CU16: Buscar anuncios con filtros combinados
Actores	Usuario Invitado, Usuario Autenticado
Descripción	Permite filtrar el catálogo de alojamientos según ciudad, rango de precio, tipo de habitación y servicios incluidos
Precondiciones	Que existan anuncios publicados y activos en la plataforma
Flujo básico	1. El usuario accede al explorador del catálogo e introduce los criterios de búsqueda. 2. El sistema consulta la base de datos aplicando los filtros de forma acumulativa. 3. El sistema devuelve los resultados paginados y actualiza la lista visual.

Tabla 3.16: Especificación del caso de uso CU16.

Caso de Uso	CU17: Explorar anuncios en mapa interactivo
Actores	Usuario Invitado, Usuario Autenticado, OpenStreetMap / Nominatim
Descripción	Muestra los anuncios disponibles mediante marcadores geográficos sobre un mapa interactivo
Precondiciones	Disponer de conectividad con el servicio cartográfico
Flujo básico	1. El usuario activa la vista de mapa en el catálogo de alojamientos. 2. El cliente solicita los marcadores comprendidos dentro del área geográfica visible. 3. El sistema renderiza las posiciones y permite interactuar con las tarjetas emergentes.

Tabla 3.17: Especificación del caso de uso CU17.

Caso de Uso	CU18: Consultar detalle de anuncio y habitaciones
Actores	Usuario Invitado, Usuario Autenticado
Descripción	Presenta la ficha completa de un anuncio, su galería gráfica, normas, fianza y habitaciones disponibles
Precondiciones	Que el anuncio seleccionado exista y se encuentre en estado visible
Flujo básico	1. El usuario pulsa sobre la tarjeta de un alojamiento desde la lista o el mapa. 2. El sistema recupera los datos del inmueble, equipamiento, perfil del anfitrión y reseñas. 3. La interfaz despliega la información organizada en pestañas de detalle.

Tabla 3.18: Especificación del caso de uso CU18.

Caso de Uso	**CU19: Consultar historial de búsquedas**
Actores	Usuario Autenticado
Descripción	Muestra los términos, rangos de precios y ubicaciones consultadas recientemente
Precondiciones	Mantener una sesión activa con actividad previa registrada
Flujo básico	1. El usuario interactúa con la barra de búsqueda o el panel de filtros. 2. El sistema recupera el registro cronológico de consultas previas del usuario. 3. La interfaz presenta sugerencias directas basadas en las preferencias recientes.

Tabla 3.19: Especificación del caso de uso CU19.

Caso de Uso	**CU20: Obtener recomendaciones personalizadas**
Actores	Usuario Autenticado
Descripción	Extiende la búsqueda de anuncios calculando sugerencias acordes al perfil y búsquedas previas
Precondiciones	Disponer de un historial de navegación con al menos una consulta registrada
Flujo básico	1. El usuario accede a la sección de recomendaciones en la pantalla de inicio. 2. El motor de afinidad analiza las búsquedas guardadas y las características del perfil. 3. El sistema ejecuta una búsqueda extendida y devuelve los alojamientos con mayor coincidencia.

Tabla 3.20: Especificación del caso de uso CU20.

Caso de Uso	**CU21: Ver reseñas y puntuación media**
Actores	Usuario Invitado, Usuario Autenticado
Descripción	Consulta las opiniones, comentarios y calificaciones otorgadas a un alojamiento
Precondiciones	Que la publicación cuente con valoraciones registradas
Flujo básico	1. El usuario accede a la pestaña de opiniones dentro de la ficha de detalle. 2. El sistema recupera las reseñas aprobadas y desglosa la media por categorías. 3. La interfaz muestra los comentarios ordenados cronológicamente con la firma del autor.

Tabla 3.21: Especificación del caso de uso CU21.

Caso de Uso	CU22: Verificar elegibilidad para reseñar
Actores	Inquilino
Descripción	Comprueba si el inquilino ha completado una estancia formalmente reservada en el inmueble
Precondiciones	Estar autenticado en el sistema
Flujo básico	1. El sistema intercepta el intento de emitir una valoración sobre un alojamiento. 2. El backend audita el histórico de reservas confirmadas y finalizadas del usuario. 3. El sistema comprueba que no exista una valoración previa emitida por la misma estancia. 4. Si la comprobación es favorable, se autoriza la apertura del formulario de reseña.

Tabla 3.22: Especificación del caso de uso CU22.

Caso de Uso	CU23: Publicar reseña y calificación
Actores	Inquilino
Descripción	Registra una valoración numérica de uno a cinco y un comentario descriptivo sobre la estancia
Precondiciones	Haber superado la verificación de elegibilidad previa
Flujo básico	1. El inquilino completa las puntuaciones desglosadas y redacta su opinión. 2. Se ejecuta de forma obligatoria la verificación de elegibilidad. 3. El sistema persiste la reseña en la base de datos y recalcula la media del alojamiento.

Tabla 3.23: Especificación del caso de uso CU23.

Gestión de inmuebles y ciclo de publicación del anfitrión

El segundo bloque abarca el registro de la vivienda física, su posicionamiento geográfico, la carga de fotos en Cloudinary y el control de publicación de las habitaciones. La figura 3.4 recoge los ocho casos de uso asociados (CU24 a CU31).

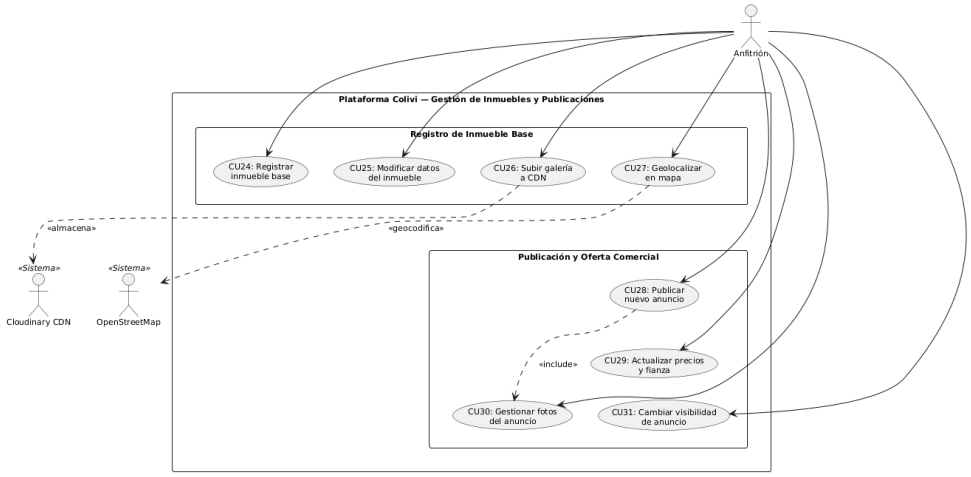


Figura 3.4: Diagrama de casos de uso de gestión de inmuebles y publicaciones del anfitrión

Las especificaciones funcionales de este ciclo se detallan desde CU24 hasta CU31:

Caso de Uso	**CU24: Registrar inmueble base**
Actores	Anfitrión
Descripción	Da de alta la estructura física de una vivienda definiendo superficie, estancias comunes y servicios
Precondiciones	Estar autenticado con perfil de anfitrión
Flujo básico	1. El anfitrión accede al asistente de registro de propiedades. 2. El anfitrión especifica la dirección, metros cuadrados, número de habitaciones y zonas comunes. 3. El sistema valida la integridad de los datos y almacena el registro del inmueble base.

Tabla 3.24: Especificación del caso de uso CU24.

Caso de Uso	**CU25: Modificar datos del inmueble base**
Actores	Anfitrión
Descripción	Permite actualizar la descripción o los equipamientos compartidos de una propiedad
Precondiciones	Ser el titular propietario del inmueble
Flujo básico	1. El anfitrión accede al panel de gestión de sus propiedades y selecciona la vivienda. 2. El anfitrión modifica las características físicas o los servicios disponibles. 3. El backend verifica la titularidad y persiste los cambios actualizados.

Tabla 3.25: Especificación del caso de uso CU25.

Caso de Uso	CU26: Subir galería fotográfica a CDN
Actores	Anfitrión, Cloudinary CDN
Descripción	Transfiere un conjunto de imágenes para asociarlas a la ficha del inmueble
Precondiciones	Archivos en formato gráfico admitido que no superen el límite de tamaño establecido
Flujo básico	1. El anfitrión selecciona una o varias imágenes desde su explorador local. 2. El cliente transfiere los archivos binarios hacia el servicio de almacenamiento seguro. 3. Cloudinary optimiza y custodia los recursos, retornando los enlaces HTTPS persistentes. 4. El backend almacena las referencias de las imágenes vinculándolas al inmueble.

Tabla 3.26: Especificación del caso de uso CU26.

Caso de Uso	CU27: Geolocalizar dirección en mapa
Actores	Anfitrión, OpenStreetMap / Nominatim
Descripción	Convierte la dirección postal introducida en coordenadas geográficas de latitud y longitud
Precondiciones	Introducir una dirección válida con calle, número, código postal y ciudad
Flujo básico	1. El anfitrión escribe la dirección postal durante el alta o edición del inmueble. 2. El cliente remite la consulta al servicio de geocodificación de OpenStreetMap. 3. El servicio devuelve las coordenadas geográficas correspondientes. 4. El sistema posiciona el marcador en el mapa y almacena los valores espaciales.

Tabla 3.27: Especificación del caso de uso CU27.

Caso de Uso	**CU28: Publicar nuevo anuncio**
Actores	Anfitrión
Descripción	Genera una oferta comercial en el catálogo vinculada a una habitación o al piso completo
Precondiciones	Disponer de un inmueble base registrado con al menos una fotografía
Flujo básico	1. El anfitrión pulsa la opción de crear publicación y selecciona el inmueble asociado. 2. El anfitrión fija el título, descripción comercial, renta mensual, fianza y fechas de disponibilidad. 3. Se invoca de forma obligatoria el caso de uso de gestión de fotografías del anuncio. 4. El sistema valida las condiciones económicas y publica la oferta en el catálogo.

Tabla 3.28: Especificación del caso de uso CU28.

Caso de Uso	**CU29: Actualizar precios, depósito y condiciones**
Actores	Anfitrión
Descripción	Permite reajustar los importes de alquiler, fianza requerida o normas de estancia
Precondiciones	Ser titular del anuncio publicado
Flujo básico	1. El anfitrión selecciona el anuncio y abre el formulario de condiciones contractuales. 2. El anfitrión modifica la renta o la fianza y confirma los cambios. 3. El sistema valida que los nuevos importes sean valores numéricos positivos. 4. El backend actualiza la oferta sin alterar las reservas previamente confirmadas.

Tabla 3.29: Especificación del caso de uso CU29.

Caso de Uso	**CU30: Seleccionar y ordenar imágenes para el anuncio**
Actores	Anfitrión
Descripción	Organiza la galería de fotografías del anuncio y define la imagen de portada principal
Precondiciones	Que el inmueble asociado disponga de imágenes cargadas en el servidor
Flujo básico	1. El anfitrión visualiza la cuadrícula de fotografías asignadas a la propiedad. 2. El anfitrión reordena las imágenes y marca la que actuará como cabecera. 3. El sistema guarda la secuencia de ordenamiento para su renderizado en catálogo.

Tabla 3.30: Especificación del caso de uso CU30.

Caso de Uso	CU31: Cambiar visibilidad de anuncio
Actores	Anfitrión
Descripción	Conmuta el estado de una publicación entre disponible, pausada o archivada
Precondiciones	Ser el propietario del anuncio
Flujo básico	1. El anfitrión selecciona la acción de cambio de estado en el panel de control. 2. El sistema actualiza el atributo de visibilidad de la publicación. 3. El catálogo omite o incluye el anuncio en los filtros públicos de forma inmediata.

Tabla 3.31: Especificación del caso de uso CU31.

3.1.5 Paquete 3: Ciclo de vida de reservas y depósitos

El paquete gestiona las propuestas de estancia sobre habitaciones activas, controlando los estados de la reserva y el depósito de garantía económica. Incorpora bloqueos de concurrencia para evitar solapamientos temporales en el calendario cuando múltiples usuarios demandan el mismo periodo. La figura 3.5 refleja la relación entre actores y los siete casos de uso del módulo (CU32 a CU38).

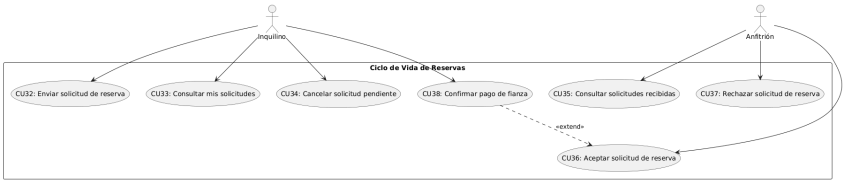


Figura 3.5: Diagrama de casos de uso del ciclo de vida de reservas

Las fichas descriptivas del ciclo de reserva comprenden desde CU32 hasta CU38:

Caso de Uso	CU32: Enviar solicitud de reserva con rango de fechas
Actores	Inquilino
Descripción	Remite una propuesta de estancia vinculada a un anuncio seleccionando fecha de entrada y salida
Precondiciones	Estar autenticado en la plataforma y no ser el titular del alojamiento
Flujo básico	1. El inquilino selecciona las fechas deseadas desde el calendario del anuncio. 2. El sistema comprueba que no existan reservas confirmadas en conflicto temporal. 3. El inquilino redacta un mensaje introductorio y confirma la solicitud. 4. El backend registra la reserva en estado pendiente y notifica al anfitrión.

Tabla 3.32: Especificación del caso de uso CU32.

Caso de Uso	CU33: Consultar historial de mis solicitudes
Actores	Inquilino
Descripción	Muestra el listado de reservas solicitadas por el usuario clasificadas por su estado actual
Precondiciones	Disponer de una sesión activa como inquilino
Flujo básico	1. El inquilino accede a su panel de reservas personales. 2. El sistema recupera el registro de solicitudes (pendientes, aceptadas, rechazadas y finalizadas). 3. La interfaz presenta los resúmenes económicos, fechas pactadas y accesos directos.

Tabla 3.33: Especificación del caso de uso CU33.

Caso de Uso	CU34: Cancelar solicitud de reserva pendiente
Actores	Inquilino
Descripción	Permite al inquilino revocar una propuesta antes del pronunciamiento del anfitrión
Precondiciones	Que la reserva se encuentre formalmente en estado pendiente de aprobación
Flujo básico	1. El inquilino selecciona la solicitud pendiente y confirma la revocación. 2. El sistema conmuta el estado de la reserva a cancelada. 3. El backend libera el bloqueo preventivo del calendario y remite aviso al propietario.

Tabla 3.34: Especificación del caso de uso CU34.

Caso de Uso	CU35: Consultar solicitudes recibidas
Actores	Anfitrión
Descripción	Despliega las peticiones de estancia pendientes de resolución para los alojamientos del propietario
Precondiciones	Ser titular de al menos un anuncio publicado
Flujo básico	1. El anfitrión accede a la bandeja de gestión de solicitudes entrantes. 2. El sistema recupera las peticiones pendientes con el perfil del demandante y su historial. 3. La interfaz expone los detalles temporales y económicos para facilitar la toma de decisión.

Tabla 3.35: Especificación del caso de uso CU35.

Caso de Uso	CU36: Aceptar solicitud de reserva
Actores	Anfitrión
Descripción	Admite formalmente la estancia de un inquilino para el rango de fechas pactado
Precondiciones	Que la solicitud se encuentre en estado pendiente y el calendario disponible
Flujo básico	1. El anfitrión examina la petición y pulsa sobre la opción de confirmación. 2. El backend bloquea de forma definitiva las fechas en el calendario del inmueble. 3. El sistema actualiza el estado a aceptada y remite notificación al inquilino.

Tabla 3.36: Especificación del caso de uso CU36.

Caso de Uso	CU37: Rechazar solicitud de reserva
Actores	Anfitrión
Descripción	Deniega una propuesta de estancia detallando de forma opcional el motivo de desestimación
Precondiciones	Que la reserva figure en estado pendiente
Flujo básico	1. El anfitrión selecciona la acción de desestimación en la solicitud correspondiente. 2. El anfitrión introduce el motivo de desestimación si lo estima oportuno. 3. El sistema marca la reserva en estado rechazada y actualiza el panel del inquilino.

Tabla 3.37: Especificación del caso de uso CU37.

Caso de Uso	CU38: Confirmar y registrar pago de fianza
Actores	Inquilino
Descripción	Registra el abono del depósito de garantía exigido por el anfitrión para consolidar el contrato
Precondiciones	Que la solicitud de reserva haya sido aceptada previamente por el anfitrión
Flujo básico	1. El inquilino accede a su reserva aceptada y pulsa la opción de pago de fianza. 2. El sistema presenta el desglose del importe del depósito establecido. 3. El inquilino introduce los datos de transacción y formaliza la operación. 4. El backend convalida la operación, emite comprobante y consolida la reserva como afianzada.

Tabla 3.38: Especificación del caso de uso CU38.

3.1.6 Paquete 4: Comunidad de convivencia y espacio del hogar

El paquete estructura la convivencia tras confirmarse el arrendamiento. Permite unir nuevos compañeros mediante códigos dinámicos de un solo uso, personalizar avatares, registrar eventos en un muro de actividad inmutable y tramitar expulsiones o bajas voluntarias siempre que no existan deudas abiertas. La figura 3.6 modela los once casos de uso que componen este espacio compartido (CU39 a CU49).

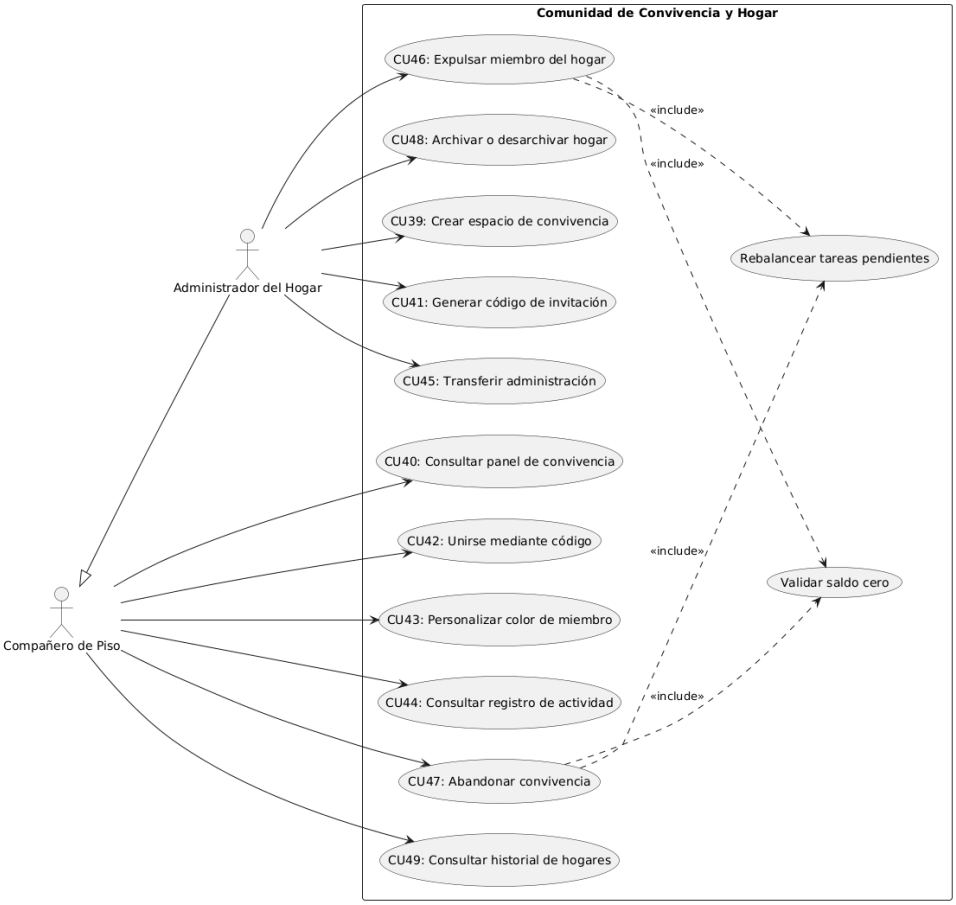


Figura 3.6: Diagrama de casos de uso de comunidad de convivencia y hogar

Las siguientes especificaciones describen los casos de uso asociados al hogar (CU39 a CU49):

Caso de Uso	CU39: Crear espacio de convivencia
Actores	Administrador del Hogar
Descripción	Da de alta un nuevo grupo de piso asignando nombre, dirección y reglas básicas
Precondiciones	Estar autenticado y no pertenecer a otro hogar en calidad de administrador simultáneo
Flujo básico	1. El usuario accede al formulario de creación de comunidad e introduce los datos del piso. 2. El sistema valida la información e inicializa el espacio de convivencia. 3. El backend otorga al creador el rol de administrador del hogar. 4. El sistema redirige al panel principal del nuevo hogar.

Tabla 3.39: Especificación del caso de uso CU39

Caso de Uso	CU40: Consultar panel y detalle de convivencia
Actores	Compañero de Piso
Descripción	Muestra la vista global del piso con la lista de miembros, balances rápidos y avisos
Precondiciones	Pertenecer activamente al hogar consultado
Flujo básico	1. El usuario accede a la sección de inicio del piso en la interfaz. 2. El sistema recupera los datos de los integrantes, sus roles y estados asignados. 3. La interfaz renderiza el resumen consolidado de convivencia y accesos a módulos.

Tabla 3.40: Especificación del caso de uso CU40

Caso de Uso	CU41: Generar o regenerar código de invitación
Actores	Administrador del Hogar
Descripción	Crea o renueva una clave alfanumérica única para la incorporación de compañeros
Precondiciones	Poseer el rol de administrador en el hogar activo
Flujo básico	1. El administrador accede a la sección de invitaciones del piso. 2. El administrador solicita la emisión o sustitución del código de acceso. 3. El backend genera una cadena alfanumérica unívoca e invalida la anterior. 4. La interfaz presenta la clave lista para ser compartida por canales externos.

Tabla 3.41: Especificación del caso de uso CU41

Caso de Uso	CU42: Unirse a un hogar mediante código
Actores	Compañero de Piso
Descripción	Permite a un usuario vincularse a un piso introduciendo la clave de invitación
Precondiciones	Estar autenticado en el sistema y disponer de un código de invitación válido
Flujo básico	1. El usuario introduce el código alfanumérico en la pantalla de bienvenida o unirse. 2. El backend verifica la autenticidad y vigencia de la clave introducida. 3. El sistema añade al usuario como compañero activo del hogar. 4. El sistema notifica la incorporación en el muro general de actividad.

Tabla 3.42: Especificación del caso de uso CU42

Caso de Uso	CU43: Personalizar color de identificación
Actores	Compañero de Piso
Descripción	Configura la tonalidad cromática asignada al usuario para calendarios y balances
Precondiciones	Ser miembro activo del piso
Flujo básico	1. El compañero abre el selector de color en la configuración de su tarjeta de miembro. 2. El compañero selecciona una tonalidad disponible en la paleta del hogar. 3. El backend persiste la preferencia cromática asociada a la membresía. 4. La interfaz actualiza los distintivos del usuario en tareas y cuadrantes.

Tabla 3.43: Especificación del caso de uso CU43

Caso de Uso	CU44: Consultar registro cronológico de actividad
Actores	Compañero de Piso
Descripción	Despliega el histórico auditable de sucesos relevantes acontecidos en el hogar
Precondiciones	Pertenecer al hogar correspondiente
Flujo básico	1. El compañero accede a la pestaña de actividad reciente del piso. 2. El sistema recupera los registros ordenados cronológicamente de forma descendente. 3. La interfaz visualiza eventos de gastos, liquidaciones, tareas y cambios de miembros.

Tabla 3.44: Especificación del caso de uso CU44

Caso de Uso	CU45: Transferir privilegios de administrador
Actores	Administrador del Hogar
Descripción	Traspasa la condición de administrador del espacio a otro compañero de piso
Precondiciones	Ser el administrador vigente y seleccionar un miembro activo diferente
Flujo básico	1. El administrador elige a un compañero y pulsa la opción de cesión de privilegios. 2. El sistema solicita confirmación formal para prevenir transferencias involuntarias. 3. El backend reasigna los roles de membresía en la base de datos de manera atómica. 4. El sistema actualiza los permisos de acceso y emite un evento en el feed del hogar.

Tabla 3.45: Especificación del caso de uso CU45

Caso de Uso	CU46: Expulsar miembro del hogar
Actores	Administrador del Hogar
Descripción	Da de baja forzosa a un integrante del piso tras validar la liquidación de sus saldos
Precondiciones	Que el integrante a expulsar no ostente deudas ni créditos pendientes
Flujo básico	1. El administrador selecciona la acción de expulsión sobre el integrante. 2. El sistema verifica obligatoriamente que el saldo neto del integrante sea cero. 3. El sistema ejecuta el rebalanceo automático de las tareas domésticas pendientes. 4. El backend conmuta el estado de la membresía a inactiva y revoca sus accesos al piso.

Tabla 3.46: Especificación del caso de uso CU46

Caso de Uso	CU47: Abandonar convivencia
Actores	Compañero de Piso
Descripción	Tramita la baja voluntaria del usuario del espacio de convivencia
Precondiciones	Mantener un balance financiero equilibrado en cero y no ser el único administrador
Flujo básico	1. El compañero selecciona la opción de abandonar el hogar desde los ajustes. 2. El sistema comprueba de forma obligatoria que su balance contable sea exactamente cero. 3. El sistema reasigna de manera automática las tareas domésticas que tenía asignadas. 4. El backend desvincula al usuario y archiva su registro histórico en el hogar.

Tabla 3.47: Especificación del caso de uso CU47

Caso de Uso	CU48: Archivar o desarchivar vista de hogar
Actores	Administrador del Hogar
Descripción	Oculta la visualización activa de un piso concluido sin eliminar sus datos históricos
Precondiciones	Que no existan estancias en curso ni deudas monetarias abiertas
Flujo básico	1. El administrador activa la conmutación de archivado en la configuración del piso. 2. El sistema comprueba el cierre satisfactorio de todos los compromisos comunes. 3. El backend traslada el hogar al archivo histórico de los participantes.

Tabla 3.48: Especificación del caso de uso CU48

Caso de Uso	CU49: Consultar historial de hogares pasados
Actores	Administrador del Hogar, Compañero de Piso
Descripción	Permite revisar datos y registros de pisos en los que se participó anteriormente
Precondiciones	Haber formado parte de al menos una convivencia finalizada o archivada
Flujo básico	1. El usuario accede a la sección de archivos de convivencia en su panel personal. 2. El sistema recupera los expedientes históricos en modo de solo lectura. 3. La interfaz presenta los resúmenes de estancias, compañeros y balances consolidados.

Tabla 3.49: Especificación del caso de uso CU49

3.1.7 Paquete 5: Gestión financiera compartida

El paquete resuelve la contabilidad compartida entre compañeros de piso. Registra facturas con repartos proporcionales, simplifica grafos de deuda para minimizar transferencias cruzadas y almacena recibos de liquidación directa. La figura 3.7 detalla las relaciones con los siete casos de uso financieros (CU50 a CU56).

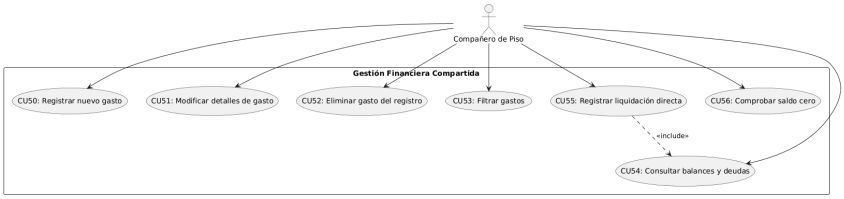


Figura 3.7: Diagrama de casos de uso de gestión financiera compartida

Las fichas tabulares desglosan los casos de uso CU50 a CU56:

Caso de Uso	**CU50: Registrar nuevo gasto con reparto**
Actores	Compañero de Piso
Descripción	Introduce un gasto abonado por un miembro distribuyendo el coste entre los integrantes
Precondiciones	Ser miembro activo del hogar con al menos otro compañero vinculado
Flujo básico	1. El compañero accede al formulario de gastos e introduce importe, concepto y fecha. 2. El compañero selecciona los participantes asignados al reparto de la cuota. 3. El sistema valida la consistencia matemática entre el total y los importes desglosados. 4. El backend registra la transacción contable y actualiza de inmediato los saldos consolidados.

Tabla 3.50: Especificación del caso de uso CU50

Caso de Uso	**CU51: Modificar detalles o reparto de gasto**
Actores	Compañero de Piso
Descripción	Permite actualizar la cuantía, concepto o miembros implicados en un gasto registrado
Precondiciones	Ser el creador del registro del gasto y que no pertenezca a un período cerrado
Flujo básico	1. El compañero localiza el asiento contable y pulsa sobre la opción de edición. 2. El compañero corrige el importe o reconfigura los miembros que participan. 3. El sistema recalcula las diferencias deudoras y acreedoras asociadas. 4. El backend actualiza la entidad persistida y ajusta los balances generales del hogar.

Tabla 3.51: Especificación del caso de uso CU51

Caso de Uso	**CU52: Eliminar gasto del registro**
Actores	Compañero de Piso
Descripción	Suprime un gasto registrado previamente revirtiendo su impacto en los balances
Precondiciones	Ostentar la autoría del apunte contable
Flujo básico	1. El compañero selecciona el gasto y solicita su retirada formal del historial. 2. El sistema pide confirmación explícita indicando el ajuste contable consecuente. 3. El backend anula la imputación de deuda y restituye los saldos previos de cada miembro. 4. El sistema notifica la supresión del registro en el panel de actividad del piso.

Tabla 3.52: Especificación del caso de uso CU52

Caso de Uso	CU53: Filtrar gastos por fechas y participantes
Actores	Compañero de Piso
Descripción	Permite segmentar el histórico de gastos según intervalos de tiempo o miembros
Precondiciones	Pertenecer al espacio de convivencia consultado
Flujo básico	1. El usuario accede a la pestaña de histórico contable en el panel de gastos. 2. El usuario define un rango de fechas, un pagador concreto o un estado de liquidación. 3. El sistema consulta las transacciones coincidentes y devuelve la lista paginada. 4. La interfaz recalcula y muestra la suma total de las operaciones filtradas.

Tabla 3.53: Especificación del caso de uso CU53

Caso de Uso	CU54: Consultar balances y transferencias sugeridas
Actores	Compañero de Piso
Descripción	Muestra el estado neto de saldos y la propuesta óptima de pagos para saldar deudas
Precondiciones	Estar formalmente integrado en el hogar
Flujo básico	1. El compañero entra en la vista de balances compartidos. 2. El sistema procesa la matriz de deudas aplicando el algoritmo de simplificación de transferencias. 3. La interfaz visualiza a quién debe dinero cada miembro o cuánto debe percibir. 4. El sistema expone las transacciones sugeridas con importes mínimos para saldar cuentas.

Tabla 3.54: Especificación del caso de uso CU54

Caso de Uso	**CU55: Registrar liquidación directa**
Actores	Compañero de Piso
Descripción	Asienta un pago directo efectuado entre dos compañeros para compensar una deuda
Precondiciones	Mantener un saldo deudor pendiente respecto al destinatario de la compensación
Flujo básico	1. El deudor accede a la lista de transferencias sugeridas y pulsa liquidar saldo. 2. Se invoca de forma obligatoria la consulta de balances y transferencias sugeridas. 3. El usuario confirma el importe transferido y el método de pago utilizado. 4. El backend asienta la liquidación y actualiza los saldos de ambos compañeros a cero.

Tabla 3.55: Especificación del caso de uso CU55

Caso de Uso	**CU56: Comprobar saldo cero**
Actores	Compañero de Piso
Descripción	Valida automáticamente que un miembro no posea deudas activas ni acreencias en el piso
Precondiciones	Estar en proceso de abandono o expulsión del espacio de convivencia
Flujo básico	1. El sistema intercepta una solicitud de baja voluntaria o expulsión de un miembro. 2. El backend computa el balance neto acumulado del usuario en todos los gastos registrados. 3. Si el saldo es exactamente cero, el sistema confirma la validación favorablemente. 4. Si existe balance positivo o negativo, el sistema bloquea la operación e indica la liquidación previa.

Tabla 3.56: Especificación del caso de uso CU56

3.1.8 Paquete 6: Motor de tareas domésticas y sistema de puntos

El paquete automatiza la asignación y supervisión de las tareas del hogar. Implementa un motor de rotación equitativa (*Round-Robin*), un calendario compartido de turnos, incentivos gamificados mediante puntuaciones y una función de rescate para tareas vencidas. La figura 3.8 expone los siete casos de uso del subsistema (CU57 a CU63).

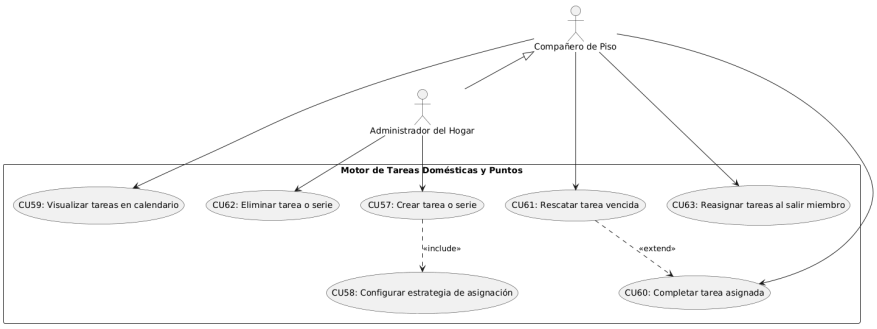


Figura 3.8: Diagrama de casos de uso de tareas domésticas y puntos

Las siguientes fichas estructuran el comportamiento de CU57 a CU63:

Caso de Uso	**CU57: Crear tarea única o serie recurrente**
Actores	Administrador del Hogar
Descripción	Da de alta una labor doméstica puntual o una serie periódica vinculada a un valor en puntos
Precondiciones	Ostentar el rol de administrador en el hogar correspondiente
Flujo básico	1. El administrador accede al planificador de tareas y pulsa sobre la opción de nueva labor. 2. El administrador define el título, descripción, periodicidad, fecha límite y recompensa en puntos. 3. Se invoca de forma obligatoria la configuración de la estrategia de asignación de miembros. 4. El backend genera la serie o tarea única y la incorpora al calendario general del piso.

Tabla 3.57: Especificación del caso de uso CU57

Caso de Uso	**CU58: Configurar estrategia de asignación**
Actores	Administrador del Hogar
Descripción	Determina la política de adjudicación de turnos entre fija o rotativa equitativa
Precondiciones	Estar en proceso de creación o parametrización de una labor doméstica
Flujo básico	1. El administrador elige si la tarea recae siempre en un miembro fijo o en una secuencia cíclica. 2. En caso rotativo, el sistema ordena a los compañeros activos para repartir la carga. 3. El motor de planificación almacena el orden de turnos asociado a la serie.

Tabla 3.58: Especificación del caso de uso CU58

Caso de Uso	CU59: Visualizar tareas en calendario y cuadrante
Actores	Compañero de Piso
Descripción	Muestra el estado de las labores domésticas en formato de cuadrante mensual o lista diaria
Precondiciones	Formar parte activa del espacio de convivencia
Flujo básico	1. El compañero abre el panel de tareas domésticas en la interfaz. 2. El sistema recupera las tareas programadas con su estado (pendiente, completada o vencida). 3. La interfaz renderiza las labores diferenciando a los asignatarios por su código de color.

Tabla 3.59: Especificación del caso de uso CU59

Caso de Uso	CU60: Completar tarea asignada
Actores	Compañero de Piso
Descripción	Marca una labor asignada como finalizada sumando los puntos previstos al perfil
Precondiciones	Que la tarea figure asignada al usuario y no se encuentre ya convalidada
Flujo básico	1. El compañero selecciona su tarea pendiente en el cuadrante y confirma su realización. 2. El backend marca la labor en estado completada y registra la marca de tiempo de cierre. 3. El sistema añade los puntos de recompensa al balance de gamificación del usuario. 4. Si la tarea formaba parte de una serie rotativa, el sistema planifica la siguiente iteración.

Tabla 3.60: Especificación del caso de uso CU60

Caso de Uso	CU61: Rescatar tarea vencida de otro miembro
Actores	Compañero de Piso
Descripción	Asume y completa una labor fuera de plazo no realizada por otro compañero sumando puntos extra
Precondiciones	Que la labor se encuentre formalmente vencida y asignada originalmente a otro compañero
Flujo básico	1. El compañero localiza una tarea expirada en el tablón común y activa la opción de rescate. 2. El sistema transfiere de forma transitoria la asignación de la labor al nuevo usuario. 3. Se invoca de forma extendida el caso de uso de completar tarea asignada. 4. El sistema asigna la bonificación de puntos al rescatador y emite notificación al piso.

Tabla 3.61: Especificación del caso de uso CU61

Caso de Uso	CU62: Eliminar tarea o serie recurrente
Actores	Administrador del Hogar
Descripción	Da de baja una tarea programada o cancela de manera definitiva una serie recurrente
Precondiciones	Poseer privilegios de administración en el espacio doméstico
Flujo básico	1. El administrador selecciona la labor a suprimir y elige entre borrar la instancia o la serie entera. 2. El sistema solicita confirmación advirtiendo sobre las asignaciones futuras afectadas. 3. El backend purga los registros planificados y recalcula el calendario del hogar.

Tabla 3.62: Especificación del caso de uso CU62

Caso de Uso	CU63: Reasignar tareas al salir un miembro
Actores	Compañero de Piso, Administrador del Hogar
Descripción	Redistribuye automáticamente las labores que estaban asignadas a un miembro saliente
Precondiciones	Que un miembro abandone el piso o sea expulsado formalmente
Flujo básico	1. El sistema intercepta el proceso de desvinculación formal de un integrante. 2. El backend audita todas las tareas pendientes o futuras asociadas al usuario saliente. 3. El motor de rotación redistribuye las responsabilidades entre los miembros restantes. 4. El sistema notifica la actualización del cuadrante a los compañeros activos.

Tabla 3.63: Especificación del caso de uso CU63

3.1.9 Paquete 7: Mensajería contextual

El canal de mensajería vincula cada hilo de chat con el anuncio correspondiente. Permite que inquilinos y propietarios intercambien mensajes antes o después de reservar, manteniendo el histórico de comunicaciones, el estado de lectura y la trazabilidad necesaria ante disputas. La figura 3.9 esquematiza los cinco casos de uso del módulo (CU64 a CU68).



Figura 3.9: Diagrama de casos de uso de mensajería contextual

Las siguientes tablas recogen la especificación de CU64 a CU68:

Caso de Uso	CU64: Iniciar conversación vinculada a un anuncio
Actores	Usuario Autenticado
Descripción	Abre un nuevo hilo de chat directo entre el usuario y el anfitrión de una publicación
Precondiciones	Estar autenticado, que el anuncio esté activo y no ser el propio propietario del mismo
Flujo básico	1. El usuario accede a la ficha de un alojamiento y pulsa sobre la opción de contactar. 2. El sistema comprueba si ya existía una conversación previa abierta para ese contexto. 3. Si existe, redirige al hilo histórico; en caso contrario, inicializa un nuevo canal vinculado. 4. La interfaz abre la ventana de conversación lista para la redacción de mensajes.

Tabla 3.64: Especificación del caso de uso CU64

Caso de Uso	CU65: Consultar bandeja de entrada y no leídos
Actores	Usuario Autenticado
Descripción	Despliega la relación de hilos de mensajería del usuario con indicadores de mensajes pendientes
Precondiciones	Mantener una sesión autenticada activa
Flujo básico	1. El usuario accede a la sección de mensajes desde la barra superior de navegación. 2. El sistema recupera las conversaciones activas ordenadas por la fecha del último mensaje. 3. La interfaz presenta los resúmenes de cada chat, los avatares y el recuento de no leídos.

Tabla 3.65: Especificación del caso de uso CU65

Caso de Uso	CU66: Ver historial de mensajes
Actores	Usuario Autenticado
Descripción	Carga y visualiza la secuencia cronológica de intervenciones dentro de una conversación
Precondiciones	Ser participante legítimo (emisor o receptor) de la conversación seleccionada
Flujo básico	1. El usuario selecciona un hilo de conversación de su bandeja de entrada. 2. El backend verifica la pertenencia del usuario al hilo solicitado. 3. El sistema devuelve los mensajes paginados con sus marcas de tiempo asociadas. 4. La interfaz renderiza el diálogo diferenciando visualmente los mensajes propios y ajenos.

Tabla 3.66: Especificación del caso de uso CU66

Caso de Uso	CU67: Enviar mensaje de texto
Actores	Usuario Autenticado
Descripción	Transmite un nuevo mensaje de texto plano hacia el destinatario en el hilo activo
Precondiciones	Formar parte de la conversación y que el contenido no supere los límites de longitud
Flujo básico	1. El usuario introduce el texto en el área de entrada del chat y pulsa enviar. 2. El cliente valida que el cuerpo del mensaje no se encuentre vacío. 3. El backend persiste el mensaje en base de datos con la marca temporal actual. 4. El sistema notifica al receptor y actualiza la vista de la conversación en tiempo real.

Tabla 3.67: Especificación del caso de uso CU67

Caso de Uso	CU68: Marcar conversación como leída
Actores	Usuario Autenticado
Descripción	Actualiza el estado de los mensajes entrantes poniendo a cero los indicadores pendientes
Precondiciones	Tener mensajes recibidos no leídos en el hilo visualizado
Flujo básico	1. El usuario abre o mantiene en foco una conversación con mensajes pendientes. 2. El cliente emite una petición asíncrona informando de la visualización del hilo. 3. El backend actualiza la marca de lectura de los mensajes correspondientes. 4. La interfaz disminuye o suprime los distintivos numéricos de notificación global.

Tabla 3.68: Especificación del caso de uso CU68

3.1.10 Paquete 8: Asistente inteligente y orquestación de herramientas

El asistente virtual responde consultas en lenguaje natural apoyándose en Spring AI y en un servidor MCP (*Model Context Protocol*). Permite filtrar habitaciones, consultar el estado de las tareas del hogar, resumir la mensajería entrante e inspeccionar colas de moderación. Para proteger los datos, el backend expide tickets efímeros con permisos de solo lectura antes de invocar las herramientas MCP. La figura 3.10 expone los siete casos de uso del asistente (CU69 a CU75).

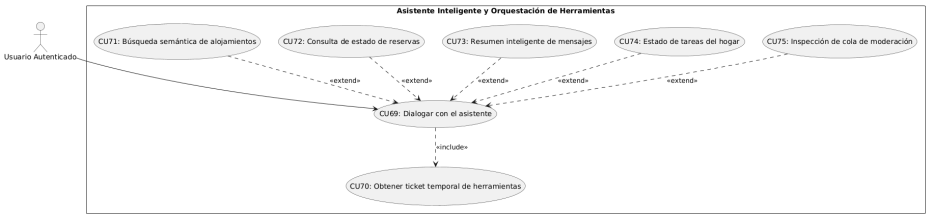


Figura 3.10: Diagrama de casos de uso de asistente inteligente y orquestación de herramientas

Las siguientes tablas detallan los casos de uso del agente inteligente (CU69 a CU75):

Caso de Uso	**CU69: Dialogar con el asistente en lenguaje natural**
Actores	Usuario Autenticado
Descripción	Mantiene una conversación interactiva con el agente inteligente mediante lenguaje natural
Precondiciones	Estar autenticado con una sesión activa en la aplicación
Flujo básico	1. El usuario abre el componente flotante de chat con el asistente e introduce una consulta. 2. Se ejecuta de forma obligatoria la expedición de un ticket temporal de autorización para herramientas. 3. El orquestador del backend procesa la petición evaluando si requiere invocar herramientas especializadas. 4. El modelo genera la respuesta estructurada y la interfaz la visualiza mediante transmisión progresiva.

Tabla 3.69: Especificación del caso de uso CU69

Caso de Uso	**CU70: Obtener ticket temporal de autenticación de herramientas**
Actores	Usuario Autenticado
Descripción	Emite una credencial temporal firmada para autorizar llamadas seguras al servidor de herramientas MCP
Precondiciones	Mantener un token de sesión JWT válido
Flujo básico	1. El subsistema de inteligencia artificial intercepta la interacción del usuario autenticado. 2. El servicio emite un ticket criptográfico de corta duración vinculado al identificador del usuario. 3. El backend traslada el ticket en las cabeceras de comunicación con el servidor MCP. 4. El servidor MCP convalida la firma del ticket garantizando el aislamiento de privilegios.

Tabla 3.70: Especificación del caso de uso CU70

Caso de Uso	CU71: Herramienta: Búsqueda semántica por preferencias
Actores	Usuario Autenticado
Descripción	Recupera anuncios considerando el estilo de vida, afinidad y descriptores cualitativos del usuario
Precondiciones	Que la consulta del usuario exprese requerimientos de convivencia o preferencias ambientales
Flujo básico	1. El modelo identifica la intención de localización cualitativa durante el diálogo con el asistente. 2. El agente invoca de forma extendida la herramienta de búsqueda semántica con los criterios extraídos. 3. El servidor MCP consulta el catálogo indexado y devuelve los alojamientos con mayor afinidad contextual. 4. El asistente sintetiza la respuesta recomendando los inmuebles más adecuados en la conversación.

Tabla 3.71: Especificación del caso de uso CU71

Caso de Uso	CU72: Herramienta: Consulta de estado de reservas
Actores	Usuario Autenticado
Descripción	Comprueba y resume el estado actual de las solicitudes de reserva emitidas o recibidas
Precondiciones	Que el usuario pregunte expresamente sobre la situación de sus estancias o reservas
Flujo básico	1. El modelo detecta una petición sobre el progreso de las reservas en curso. 2. El agente ejecuta la llamada a la herramienta MCP transmitiendo el contexto de identidad del usuario. 3. El servicio consulta la base de datos y compila las reservas pendientes, aceptadas o próximas a expirar. 4. El asistente traslada un resumen conciso indicando las acciones pendientes que requiere cada solicitud.

Tabla 3.72: Especificación del caso de uso CU72

Caso de Uso	CU73: Herramienta: Resumen inteligente de mensajes
Actores	Usuario Autenticado
Descripción	Sintetiza los temas clave y mensajes no leídos acumulados en la bandeja de entrada
Precondiciones	Disponer de mensajes recibidos en las conversaciones de la plataforma
Flujo básico	1. El usuario solicita al asistente conocer las novedades de sus conversaciones activas. 2. El agente ejecuta condicionalmente la herramienta de agregación y análisis de mensajería. 3. El servidor MCP recopila los hilos recientes omitiendo información confidencial no pertinente. 4. El modelo estructura un extracto de las comunicaciones pendientes organizadas por remitente y urgencia.

Tabla 3.73: Especificación del caso de uso CU73

Caso de Uso	CU74: Herramienta: Estado de tareas del hogar
Actores	Usuario Autenticado
Descripción	Proporciona una panorámica rápida sobre las tareas domésticas pendientes y las fechas límite en el piso
Precondiciones	Pertenecer activamente a un hogar constituido
Flujo básico	1. El usuario consulta mediante lenguaje natural por las labores domésticas que tiene pendientes. 2. El asistente activa la herramienta MCP correspondiente consultando el módulo del hogar del usuario. 3. El servicio devuelve las tareas asignadas para la semana, turnos rotativos y tareas susceptibles de rescate. 4. El modelo expone el estado de las labores recordando los puntos asignados a cada una.

Tabla 3.74: Especificación del caso de uso CU74

Caso de Uso	CU75: Herramienta: Inspección de cola de moderación
Actores	Usuario Autenticado
Descripción	Asiste en la revisión y diagnóstico rápido de reportes pendientes acumulados en la cola de incidencias
Precondiciones	Poseer el rol de administración del sistema (*ADMIN*)
Flujo básico	1. Un administrador del sistema consulta al asistente sobre la carga actual de denuncias sin resolver. 2. El agente comprueba los privilegios de administración del emisor y llama a la herramienta interna MCP. 3. El servicio recopila las infracciones más graves y los usuarios reincidentes en la cola de moderación. 4. El asistente sintetiza los expedientes prioritarios facilitando la toma de decisiones sancionadoras.

Tabla 3.75: Especificación del caso de uso CU75

3.1.11 Paquete 9: Panel de administración, gobernanza y moderación global

El paquete gestiona las funciones disciplinarias y la mediación en disputas dentro de la plataforma. Cualquier usuario puede denunciar comportamientos contrarios a las normas de convivencia, mientras que los administradores revisan chats notificados, resuelven quejas en bloque y sancionan infractores inhabilitando cuentas o retirando anuncios. La figura 3.11 reúne los once casos de uso del módulo (CU76 a CU86).

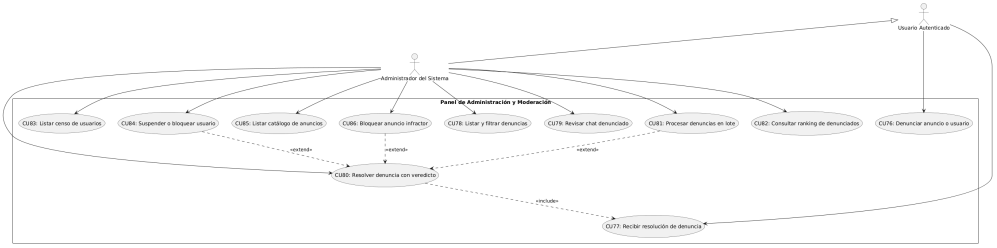


Figura 3.11: Diagrama de casos de uso de moderación y administración global

Las siguientes fichas estructuran las operaciones de moderación (CU76 a CU86):

Caso de Uso	CU76: Denunciar anuncio, usuario o conversación
Actores	Usuario Autenticado
Descripción	Remite una queja formal detallando una infracción sobre un contenido o conducta observada
Precondiciones	Estar autenticado y no haber denunciado previamente el mismo elemento si sigue en trámite
Flujo básico	1. El usuario pulsa sobre la opción de reporte presente en el anuncio, perfil o chat. 2. El usuario selecciona la categoría del motivo y aporta una descripción justificativa. 3. El backend registra la denuncia en estado pendiente dentro de la cola de moderación. 4. El sistema confirma la recepción del reporte asignando un identificador de seguimiento.

Tabla 3.76: Especificación del caso de uso CU76

Caso de Uso	CU77: Recibir notificación de resolución de denuncia
Actores	Usuario Autenticado
Descripción	Informa al usuario denunciante sobre el dictamen adoptado respecto a su notificación
Precondiciones	Haber interpuesto una denuncia que haya sido formalmente resuelta
Flujo básico	1. El sistema procesa el veredicto adoptado por el administrador sobre el expediente. 2. El backend marca la denuncia con la confirmación de retroalimentación emitida. 3. El sistema remite un aviso al panel de notificaciones del denunciante con el dictamen.

Tabla 3.77: Especificación del caso de uso CU77

Caso de Uso	CU78: Listar y filtrar cola global de denuncias
Actores	Administrador del Sistema
Descripción	Despliega las quejas registradas permitiendo su clasificación por estado, severidad o tipo
Precondiciones	Disponer del rol de administración del sistema
Flujo básico	1. El administrador accede al panel de moderación global. 2. El administrador aplica filtros por fecha, tipología del elemento o gravedad. 3. El sistema recupera el listado paginado ordenado por orden cronológico y prioridad. 4. La interfaz renderiza las tarjetas con el estado y los detalles preliminares del caso.

Tabla 3.78: Especificación del caso de uso CU78

Caso de Uso	CU79: Revisar historial de conversación denunciada
Actores	Administrador del Sistema
Descripción	Audita de manera segura los mensajes intercambiados en un hilo objeto de investigación
Precondiciones	Que la denuncia corresponda formalmente a una infracción en el contexto del chat
Flujo básico	1. El administrador pulsa sobre el enlace de inspección dentro de la ficha del reporte. 2. El sistema valida los permisos de auditoría y recupera los mensajes del canal. 3. La interfaz presenta la secuencia íntegra de intervenciones para contrastar los hechos.

Tabla 3.79: Especificación del caso de uso CU79

Caso de Uso	CU80: Resolver denuncia individual con veredicto
Actores	Administrador del Sistema
Descripción	Dictamina una queja asignando un estado de estimación o desestimación con notas técnicas
Precondiciones	Que el expediente de denuncia se encuentre en estado pendiente o en revisión
Flujo básico	1. El administrador introduce la justificación del veredicto y selecciona la resolución. 2. El backend conmuta el estado de la denuncia a resuelta archivando el motivo del fallo. 3. Se invoca de forma obligatoria la emisión de la notificación de retroalimentación al denunciante.

Tabla 3.80: Especificación del caso de uso CU80

Caso de Uso	CU81: Procesar denuncias en lote
Actores	Administrador del Sistema
Descripción	Permite resolver conjuntamente múltiples denuncias vinculadas a un mismo infractor
Precondiciones	Existencia de al menos dos quejas pendientes asociadas a la misma entidad
Flujo básico	1. El administrador marca un grupo de denuncias acumuladas sobre un mismo elemento. 2. El administrador define una resolución común y un veredicto unificado. 3. Se extiende de forma reiterada la resolución individual con dictamen para cada expediente. 4. El backend actualiza masivamente los registros dentro de una misma transacción.

Tabla 3.81: Especificación del caso de uso CU81

Caso de Uso	CU82: Consultar ranking de elementos más denunciados
Actores	Administrador del Sistema
Descripción	Despliega métricas consolidadas sobre los usuarios y publicaciones con mayor tasa de reportes
Precondiciones	Disponer de credenciales activas de administrador
Flujo básico	1. El administrador accede al panel de métricas y tendencias de moderación. 2. El sistema agrega la cantidad de reportes agrupados por identificador de entidad denunciada. 3. La interfaz presenta una tabla ordenada descendentemente destacando a infractores recurrentes.

Tabla 3.82: Especificación del caso de uso CU82

Caso de Uso	CU83: Listar usuarios registrados con filtros
Actores	Administrador del Sistema
Descripción	Muestra el censo global de cuentas en la plataforma permitiendo segmentar por estado o rol
Precondiciones	Estar autenticado con rol de administración
Flujo básico	1. El administrador entra en el módulo de gestión de usuarios del panel directivo. 2. El administrador introduce un criterio de búsqueda como correo, nombre o estado disciplinario. 3. El backend devuelve la lista paginada reflejando fechas de registro y avisos previos.

Tabla 3.83: Especificación del caso de uso CU83

Caso de Uso	CU84: Suspender o bloquear usuario de la plataforma
Actores	Administrador del Sistema
Descripción	Inhabilita temporal o definitivamente el acceso al sistema de un infractor
Precondiciones	Que la cuenta a sancionar se encuentre actualmente en estado activo
Flujo básico	1. El administrador activa la sanción sobre el perfil del infractor especificando el motivo. 2. Si la acción dimana de un informe disciplinario, se ejecuta de forma extendida sobre la resolución del reporte. 3. El backend revoca de manera inmediata todos los tókenes JWT activos del usuario. 4. El sistema bloquea los futuros intentos de autenticación impidiendo el acceso a la API.

Tabla 3.84: Especificación del caso de uso CU84

Caso de Uso	CU85: Listar catálogo global de anuncios
Actores	Administrador del Sistema
Descripción	Examina la totalidad de los alojamientos registrados con independencia de su visibilidad
Precondiciones	Disponer de privilegios de administración
Flujo básico	1. El administrador accede a la auditoría del catálogo de propiedades. 2. El sistema recupera los anuncios activos, pausados y bloqueados de todos los anfitriones. 3. La interfaz permite consultar los importes, propietarios asociados e historiales de quejas.

Tabla 3.85: Especificación del caso de uso CU85

Caso de Uso	CU86: Bloquear o despublicar anuncio infractor
Actores	Administrador del Sistema
Descripción	Retira una publicación del catálogo público por vulnerar las condiciones del servicio
Precondiciones	Que el anuncio figure en estado publicado o disponible
Flujo básico	1. El administrador selecciona la opción de bloqueo sobre la ficha del alojamiento infractor. 2. Si procede de una denuncia, la medida se extiende sobre la resolución del reporte evaluado. 3. El backend altera el estado de la publicación a bloqueada por moderación. 4. El sistema purga la oferta de los resultados de búsqueda y del mapa interactivo.

Tabla 3.86: Especificación del caso de uso CU86

4 Apartados del CUERPO DEL DOCUMENTO

El llamado “cuerpo del documento” es el grupo de apartados donde se describe como se ha desarrollado el trabajo. En el dominio de la Informática, un TFG debe mantener un equilibrio entre la componente de investigación y la componente puramente técnica asociada con el proyecto de desarrollo de software. Un projecto software conlleva la elaboración de una serie de memorias, cada una recopilando lo hecho a distintos niveles, ver figura 4.2. Sin embargo la memoria de un TFG es de carácter académico y ademas de artefactos, presupuestos y modelos debe incluir determinadas cuestiones que muestren como se han desarrollado las ideas propias. El punto de partida es la estructura típica de un articulo que se muestra en la figura 4.1.

La propuesta de estructura se define a continuación indicando que se incluiría en cada apartado. Lo importante en este caso es QUE HAY QUE INCLUIR la estructura del TFG final de cada alumno puede ser diferente, uniendo o separando los contenidos que se detallan a continuación.

Objetivos o cuestiones de investigación se establecerá el objetivo general y los específicos, es decir, cuestiones que serán respondidas con el trabajo que se va a realizar. No es imprescindible elaborar un apartado específico denominado OBJETIVOS, pues éstos puede quedar reflejados al final de la introducción

Planificación temporal Se debe incluir una referencia a como se han distribuido las tareas a lo largo del tiempo empleado en el TFG. Esta cuestión deriva de los proyectos técnicos de disciplinas de ingeniería. Puede representarse en forma de tabla o gráfico 4.3 y puede aparecer dentro del apartado de metodología.

Contextualización o estado del arte Se hará mención a los elementos conceptuales que sirven de base para la investigación, estudios previos relacionados con el problema planteado, etc. En este apartado es importante hacer un estudio de los trabajos previos o revisión bibliográfica [**? ?**]. Si como es habitual se tiene que utilizar una o varias herramientas software es en este apartado donde se recogen las alternativas consideradas y cual de ellas ha sido elegida y porqué.



Figura 4.1: Estructura clásica de un articulo de investigación



Figura 4.2: Estructura de documentos de un proyecto técnico

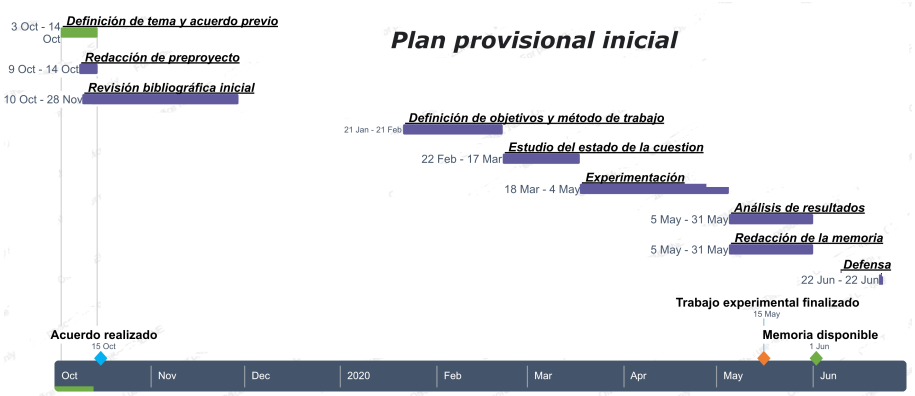


Figura 4.3: Planificación del proyecto

Metodología ¿Qué procedimiento se ha seguido para alcanzar los objetivos planteados? Se indicará el tipo o tipos de investigación, las técnicas y los procedimientos que serán utilizados para llevarla a cabo; se identificará la población y el tamaño de la muestra así como las técnicas e instrumentos de recolección de datos, si te trata de un trabajo empírico, la metodología de desarrollo de software seleccionada. O cualquier otra información sobre el proceso seguido. La planificación y objetivos también pueden estar en este apartado

Resultados Incluirá los resultados de la investigación o trabajo, así como el análisis y la discusión de los mismos, aunque algunos tutores prefieren dos apartados separados. En el caso de tratarse de un desarrollo de software en este apartado se incluirá la descripción del producto resultante y los modelos o artefactos softtware necesarios para su descripción. Pero el manual o tutorial detallado y los modelos completos deberían llevarse a los anexos.

La discusión conlleva revisar que aporta el trabajo realizado y se han conseguido los objetivos planteados inicialmente. Caso de tener una evaluación por parte de los clientes potenciales también aparecerá en este apartado.

5 Apartado BIBLIOGRAFIA

Este apartado es de vital importancia para tener éxito en la consecución del trabajo fin de grado. Los tribunales suelen prestar gran atención a este apartado, probablemente por su formación académica.

Es todo un mundo el de los estilos bibliográficos y la realización de las citas, se puede encontrar un tutorial en: `http://ci2.ual.es/comunicar-la-informacion/citas-y-referencias-bibliograficas/`. No obstante en las siguientes secciones se indicarán las cuestiones básicas para iniciarse en el tema pero restringuiendose a la utilización de la presente plantilla. La gran ventaja de latex es que cada elemento en las referencias `\bibitem` es considerado como un elemento de una base de datos, de forma que es el compilador el que reordena o coloca la cita de acuerdo con el estilo definido.

5.1 Como citar

A lo largo del texto de la memoria se incluirán citas a trabajos de otros autores. La inclusión de la cita tiene que ser justo después del texto donde se propone algo relativo al trabajo citado. Para citar, como para todo el LaTeX existen numerosos paquetes, los dos mas conocidos son biblatex y natbib .

Además dada forma de citar y de incorporar items en el apartado referencias lleva asociado un estilo de bibliografía `\bibliographystyle{XXX}`, que aun abre mucho mas el abanico de opciones.

Es recomendable consultar las ayudas de los paquetes `https://www.overleaf.com/learn/latex/Biblatex_bibliography_styles` y `https://www.overleaf.com/learn/latex/Bibliography_management_with_natbib`.

Lo habitual es utilizar la orden `\cite{xxx}`, o `\citep(xxx)` en la ubicación en el texto donde se quiere colocar la cita, el compilador se encargá de procesarla según el estilo.

5.2 Apartado referencias

Existen dos formas de incorporar las referencias, directamente en el entorno `\begin{bibliography}` y utilizando las ordenes del lenguaje o bien utilizando un archivo .bib para guardar esas referencias. Se generará un auxiliar .bbl que automáticamente se incorpora en el documento.

Este archivo auxiliar es el que tiene directamente las ordenes del lenguaje.

```
\begin{thebibliography}{3}

\bibitem{SWEBOK2014}
P.~Bourque and R.~E. Fairley, Eds., \emph{{SWEBOK}: Guide to the Software
  Engineering Body of Knowledge}, version 3.0~ed.\hskip 1em plus 0.5em minus
  0.4em\relax Los Alamitos, CA: IEEE Computer Society, 2014.

\bibitem{kitchenham_2013}
B.~Kitchenham and P.~Brereton, ``A systematic review of systematic review
  process research in software engineering,'' \emph{Information and Software
  Technology}, vol.~55, no.~12, pp. 2049 -- 2075, 2013.

\bibitem{basili1992}
V.~R. Basili, ``Software modeling and measurement: the goal/question/metric
  paradigm. cs-tr-2956, umiacs-tr-92-96,'' University of Maryland, Tech. Rep.,
  1992.
\end{thebibliography}
```

5.3 ARCHIVOS .BIB

BibTeX, y natbib por extensión, usan un archivo externo en texto plano como base de datos de referencias bibliográficas para generar las bibliografías y sus referencias en documentos con distintos formatos de artículos, libros, tesis, presentaciones, etc. Los nombres de archivos de referencias bibliográficas de BibTeX usualmente terminan o usan la extensión .bib. Los ítems bibliográficos incluidos en un .bib están separados por tipos.

```
@inbook{Berander2005,
        address = {Berlin, Heidelberg},
        author = {Berander, Patrik and Andrews, Anneliese},
        booktitle = {Engineering and Managing Software Requirements},
        pages = {69--94},
        publisher = {Springer Berlin Heidelberg},
        title = {{Requirements Prioritization}},
        doi= {https://doi.org/10.1007/3-540-28244-0{\_}4},
        year = {2005}
}
```

Los tipos que son reconocidos por virtualmente todos los estilos de BibTeX se muestran en el anexo B. Pero igual que antes es recomendable recurrir a tutoriales o recursos donde se describan con detalle. Además se recomienda la utilización de algún gestor de referencias bibliográficas tipo Mendeley. `https://www.mendeley.com/`

BIBLIOGRAFÍA

[1] Idealista Libertad y Control S.L., “Idealista: Portal inmobiliario en españa,” Recuperado 23 Octubre 2026 https://www.idealista.com, 2026.

[2] Airbnb Inc., “Airbnb: Alojamientos vacacionales y alquileres,” Recuperado 23 Octubre 2026 https://www.airbnb.es, 2026.

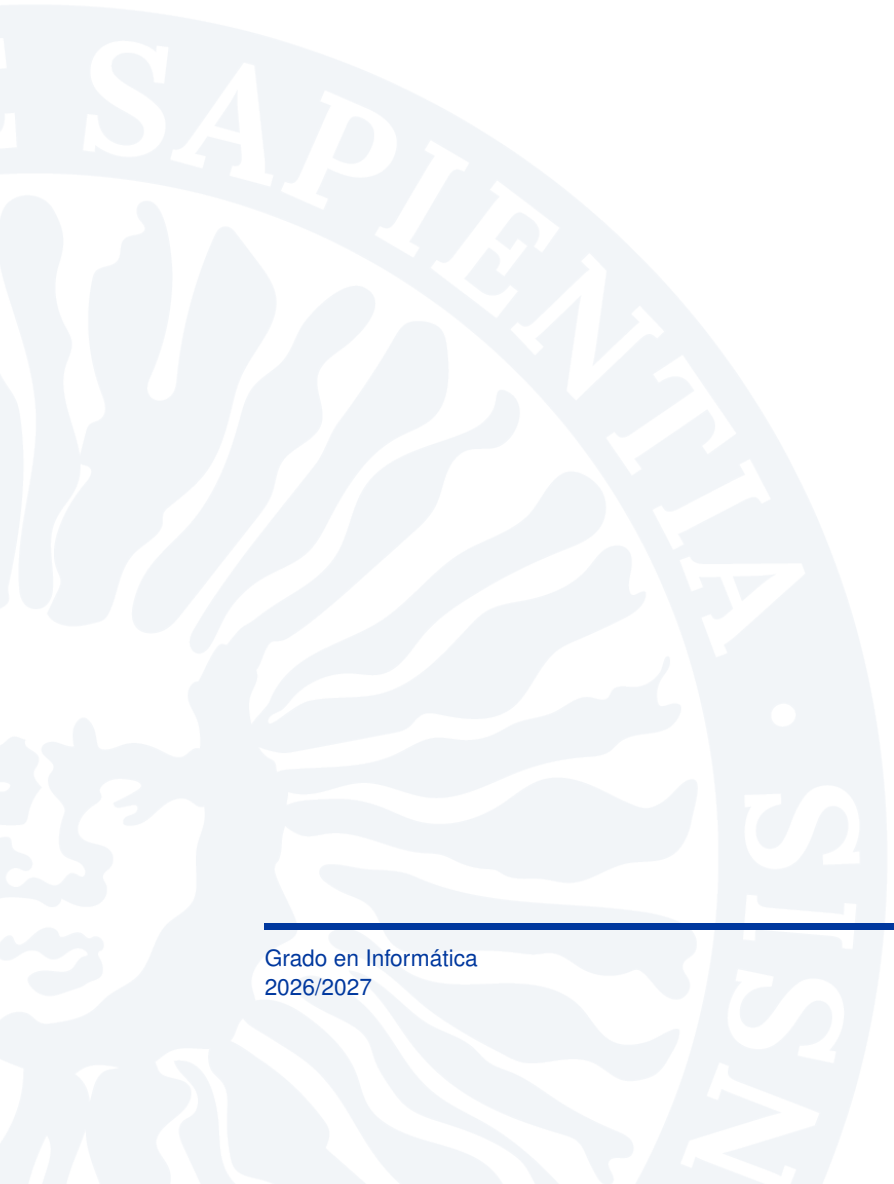
[3] Badi App S.L., “Badi: Alquiler de habitaciones y compañeros de piso,” Recuperado 23 Octubre 2026 https://badi.com, 2026.

[4] Bunq B.V., “Tricount: Gestión de gastos compartidos entre amigos,” Recuperado 23 Octubre 2026 https://www.tricount.com, 2026.

[5] Flatastic GmbH, “Flatastic: Organización doméstica y tareas de convivencia,” Recuperado 23 Octubre 2026 https://www.flatastic-app.com, 2026.

[6] P. Bourque and R. E. Fairley, Eds., *SWEBOK: Guide to the Software Engineering Body of Knowledge*. Los Alamitos, CA: IEEE Computer Society, 2014.

A Contenido de una entrada en .bib



Nombre campo	Descripción
address	Usualmente la dirección de la editorial
author	Nombre(s) del (de los) autor(es)
title	Título del libro
chapter	El número de un capítulo (o sección, etc)
edition	La edición de un libro, por ejemplo,segunda
editor	Nombre(s) del (de los) editor(es)
howpublished	Forma en que fue publicada la obra
institution	Institución responsable de un informe técnico
journal	Nombre del periódico o revista
key	Empleado para la alfabetización, referencias cruzadas y para crear una clave cuando la información del autor no está disponible. No debe confundirse con la etiqueta usada en el cite y que debe colocarse al inicio de la entrada
month	El mes de publicación o, para un trabajo inédito, en el que fue escrito
note	Cualquier información adicional que pueda ayudar al lector
number	El número del periódico, la revista, el informe técnico o del trabajo en una serie
organization	La organización responsable de una conferencia o que publica un manual
pages	Números de páginas
publisher	El nombre de la editorial. No debe confundirse con el editor
school	Nombre de la escuela donde fue escrita una tesis
series	El nombre de una serie o conjunto de libros
title	El título del trabajo
type	El tipo de un informe técnico
volume	El volumen de un periódico o una revista, o de algún libro que conste de volúmenes
year	El año de publicación. Para un trabajo inédito, el año en que fue escrito. Generalmente debe consistir de cuatro dígitos, por ejemplo 200

Tabla A.1: Campos de una entrada en archivo de bibliografía .bib

B TIPOS DE REFERENCIAS

article	Un artículo de un periódico o revista
book	Un libro con una editorial que se indica en forma explícita. Los campos requeridos en este caso son author (autor), editor, title (título), publisher (editorial) y year (año)
booklet	Una obra que está impresa y encuadernada, pero sin una editorial o institución patrocinadora
conference	Lo mismo que inproceedings, incluido para compatibilidad con el lenguaje de marcación Scribe
inbook	Una parte de un libro, que puede ser un capítulo (o sección) o un rango de páginas
incollection	Una parte de un libro que tiene su propio título
inproceedings	Un artículo en las actas de sesiones (proceedings) de una conferencia
manual	Documentación técnica
mastersthesis	Una tesis de maestría (Master thesis) o proyecto fin de carrera
misc	Para uso cuando los demás tipos no corresponden
phdthesis	Una tesis de doctorado (Ph D thesis)
proceedings	Las actas de sesiones de una conferencia
techreport	Un reporte publicado por una escuela u otra institución, usualmente numerado dentro de una serie
unpublished	Un documento que tiene un autor y título, pero que no fue formalmente publicado

Tabla B.1: Tipos posibles de elementos en bibliografía con natbib

ARTEFACTOS SOFTWARE

UNIVERSIDAD DE ALMERÍA



Resumen/Abstract

sobre Spring Boot 3 y persistencia en PostgreSQL, enlazado con un cliente web interactivo en React. A esta base técnica se añade un microservicio independiente en Node.js que implementa el protocolo MCP (*Model Context Protocol*). Gracias a esta pieza, el modelo de lenguaje consulta anuncios disponibles, inspecciona los turnos de limpieza pendientes y extrae resúmenes de la mensajería interna sin mutar datos en el servidor.

Las decisiones de diseño priorizan el desacoplamiento mediante principios SOLID y patrones de diseño guiado por el dominio (DDD). Se abordaron retos técnicos exigentes: la concurrencia en la reserva de habitaciones mediante bloqueos en base de datos, el cálculo determinista de balances contables entre compañeros y la gamificación en el reparto de tareas domésticas.

Colivi addresses the recurrent friction students face when searching for shared flats and organizing daily life with roommates. Finding rooms from abroad during exchange programs usually means dealing with fragmented listings and unverified landlords. Once settled, chores and shared expenses trigger avoidable tensions.

The architecture combines a Java 21 backend built on Spring Boot 3 with a PostgreSQL relational database and an interactive React web frontend. An external Node.js microservice exposes tools through the Model Context Protocol (MCP), enabling an LLM agent to inspect household chore rotations, query housing listings, and summarize inbox notifications safely under read-only boundaries.

Engineering decisions focused on SOLID principles and Domain-Driven Design (DDD). The implementation tackles strict database-level concurrency controls during booking transitions, graph-based debt balance resolutions between tenants, and a fair round-robin task assignment engine.

GRADO EN INGENIERÍA INFORMÁTICA

2026/2027